DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Design and Evaluation of Multi-Agentic AI Systems for Contract Lifecycle Management SaaS Platforms

Nguyen, Duc Trung, B.Sc.

DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Design and Evaluation of Multi-Agentic AI Systems for Contract Lifecycle Management SaaS Platforms

Entwurf und Evaluation multiagentenbasierter KI-Systeme für Contract-Lifecycle-Management-SaaS-Plattformen

Author:	Nguyen, Duc Trung, B.Sc.
Supervisor:	Prof. Dr. Ingo Weber
Advisor:	Samed Bayer, M.Sc.
Submission Date:	04.05.2026

Use of *AI Assistants* for Research Purposes

I have used AI Assistant(s) for the purposes of my research as part of this thesis.

Yes ☒ No ☐

Explanation: In this research, AI assistants are responsibly employed for writing assistance and code debugging. Specifically, AI tools are used to refine existing written content to align with the standards of scientific writing and to debug the implemented algorithms effectively.

I confirm in signing below, that I have reported all usage of AI Assistants for my research, and that the report is truthful and complete.

I confirm that this master's thesis in informatics is my own work and I have documented all sources and material used.

Munich, 04.05.2026 Nguyen, Duc Trung, B.Sc.

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Samed Bayer, whose close guidance shaped this thesis from first concept to final submission. His willingness to engage deeply with the research questions, provide sharp and constructive feedback, and push the work toward scientific rigour made a decisive difference. I also thank Prof. Dr. Ingo Weber for providing the academic home for this project.

I am also grateful to colleagues and friends working in industry who shared their practical experience with enterprise AI deployments. Conversations about what contract intelligence has to do in production helped me sharpen the research questions and keep the evaluation grounded.

Finally, I owe a great deal to my partner, Ngo Dan Phuong, for her patience and steady support through the long months of this work. Her encouragement carried me through the harder stretches of the project.

I would not have been able to accomplish this work without the support of each of these people.

Abstract

Contract Lifecycle Management platforms increasingly rely on large language models (LLMs) to extract clauses from commercial contracts. Vendors and recent academic systems claim that splitting the task across several specialised agents outperforms a single well-prompted model, but no published study tests this claim head-to-head across modern model families on a shared benchmark.

This thesis compares five extraction setups: zero-shot, chain-of-thought, a classify-then-extract pipeline, a router that dispatches to three domain specialists, and a single agent that consolidates the specialist prompts. Each setup is run with eleven frontier models from Anthropic, Google, and OpenAI on the 102-contract test split of the Contract Understanding Atticus Dataset (CUAD), producing 163,644 paired predictions compared with established statistical tests.

No multi-agent setup consistently beats the strongest single-agent baseline. Overall F_2 scores fall within roughly one percentage point of one another (0.790–0.801), and each provider peaks under a different configuration. Two findings explain the flat result. The pipeline's first-stage classifier silently rejects valid clauses on the Gemini models, where it accounts for nearly all false negatives. The LLM router, in turn, almost always reproduces a simple category-to-specialist lookup, so the orchestration adds little that prompt engineering does not. Multi-agent setups also cost 23–67% more per contract without a matching accuracy gain. Their one durable advantage is verbatim grounding: every predicted clause is cited from the contract, compared with about half under zero-shot prompting.

For legal teams and the vendors building their tools, one well-prompted model usually reviews contracts as accurately as a more elaborate multi-agent system. The extra agents earn their cost mainly when a regulator or client needs to see exactly where each clause came from.

Contents

Acknowledgments **iii**

Abstract **iv**

1 Introduction **1**
1.1 Motivation 1
1.2 Research Gap 1
1.3 Research Questions 2
1.4 Hypotheses 3
1.5 Contributions 3
1.6 Thesis Outline 4

2 Background and Related Work **5**
2.1 Contract Lifecycle Management 5
2.2 Large Language Models 6
2.3 Agentic AI and Multi-Agent Systems 7
2.4 The CUAD Benchmark 9
2.5 Contract Analysis Methods 10
2.6 Multi-Agent Legal AI 11
2.7 Conditions for Multi-Agent Effectiveness 13
2.8 Evaluation Methodology 13
2.9 Research Gap 16

3 Methodology **17**
3.1 Research Design 17
3.2 Dataset and Sampling 18
3.3 Model Selection 19
3.4 Metrics 20
3.5 Hypotheses 21
3.6 Statistical Methodology 22
3.7 Threats to Validity 24

4 System Design and Implementation **26**
4.1 System Components and Data Flow 26
4.2 Implementation Stack 27
4.3 Shared Abstractions 28
4.4 Baseline Architectures 29
4.5 Multi-Agent Architectures 29
4.6 Observability and Trace Schema 33
4.7 Experimental Infrastructure 34

5 Results **36**
5.1 Run Coverage and Sample Accounting 36
5.2 H1: Multi-Agent vs. Single-Agent Baselines 37
5.3 H2: Rare-Category Gap 42
5.4 H3: Architecture vs. Prompt Content (Multi-Agent (Full) vs. Multi-Agent (Prompt-Only)) 44
5.5 Classification-Gate Cascade 46
5.6 Span Grounding Rate 48
5.7 Cost-Efficiency Frontier 49
5.8 Contract-Length Moderation 51
5.9 Answers to Research Questions 52

6 Discussion **53**
6.1 The Headline Finding and Its Conditions 53
6.2 The Cascade as a Recall–Precision Exchange 54
6.3 Routing as a Deterministic Lookup 55
6.4 The β=2 Choice and Operating-Point Sensitivity 56
6.5 Benchmark-to-Production Gaps 57

7 Conclusion **59**
7.1 Summary of Contributions 59
7.2 Answers to the Research Questions 60
7.3 Limitations 62
7.4 Future Work 63

List of Figures **65**

List of Tables **67**

Bibliography **70**

1 Introduction

1.1 Motivation

Contract Lifecycle Management (CLM) is the workflow through which an organisation authors, negotiates, executes, monitors, and retires its contractual instruments. The scale of the problem is substantial: organisations forfeit value equivalent to roughly 9% of annual revenue through poor contracting practices, with contract data dispersed across an average of 24 systems and 83% of executives reporting their contracts too rigid to accommodate change [Wor25]. Goldman Sachs estimates 44% of U.S. legal-task time is technically automatable by generative AI, the second-highest exposure across all professions [Hat+23].

Frontier large language models (LLMs) match junior lawyers on issue-identification F-scores (0.87 vs. 0.88) while reducing review time by $\approx 100\times$ [Mar+24], and independent benchmarks confirm competitiveness on standardised clause-extraction tasks [Hen+21; Liu+25]. Single-prompt deployments nonetheless dominate commercial CLM, and three production failures persist: hallucination, prompt sensitivity [Cha+24], and laziness: the silent default to "no related clause" even when the clause is present in the contract [Liu+25].

Multi-agent architectures are the proposed remedy. Decomposing clause analysis into specialised agents promises higher recall through specialisation and inspectable per-decision provenance traces of the kind required under Article 13 of the EU AI Act [EC24] and Article 22 of the GDPR [EC16]. Frameworks including AutoGen [Wu+23], MetaGPT [Hon+24], and Mixture-of-Agents [Wan+24] have made such architectures straightforward to instantiate. Whether they improve contract analysis is an open empirical question.

1.2 Research Gap

Existing legal multi-agent systems (ChatLaw [Cui+24], PAKTON [Rap+25], L-MARS [WY25]) each evaluate a single architecture on a single benchmark without ablating the architec-

tural choice itself. The empirical question of which multi-agent pattern is preferable for clause-level contract analysis under a fixed model and task remains unresolved.

Gao, Y. Li, B. Liu, et al. [Gao+25] show diminishing returns for frontier models whose long-context reasoning absorbs the workload that previously required decomposition. Cemri, Pan, Yang, et al. [Cem+25] construct a 14-mode failure taxonomy across seven multi-agent frameworks and report only modest 14–15% recovery from simple interventions, suggesting structural rather than prompt-level causes. Bogavelli, Subramani, and Sharma [BSS25] find no universally optimal architecture across frontier models on enterprise tasks, with model–architecture interaction dominating absolute architectural ranking.

Prior CUAD evaluations [Liu+25] have used inconsistent sampling designs: inflated positive ratios produce optimistic precision estimates and mask over-extraction as a failure mode (§2.4).

No published study simultaneously (i) cross-varies multi-agent architecture and frontier-model family on CUAD, (ii) reports both laziness and over-extraction as paired failure-mode metrics, (iii) applies paired statistical tests with multiple-comparison correction, and (iv) ablates the multi-agent design against a consolidated single-agent prompt. This thesis addresses all four requirements.

1.3 Research Questions

Main RQ. Under what conditions, if any, do multi-agent architectures improve contract analysis accuracy, efficiency, and explainability in enterprise CLM systems, and which architectural patterns are most effective?

RQ1. Under what model-family and prevalence conditions does each architectural pattern (pipeline decomposition, specialist routing, or consolidated prompting) achieve the highest F_2?

RQ2. How do multi-agent configurations compare to single-agent baselines in F_2 and provider cost across clause categories and contract lengths?

RQ3. Which auditability properties (trace completeness, decision attribution, span grounding) does each architectural configuration enable?

1.4 Hypotheses

H1. Multi-agent configurations consistently outperform the strongest single-agent baseline on F_2 across the model panel (defined in §2.8; McNemar exact, α=0.05, Benjamini–Hochberg corrected; consistency operationalised in §3.5).

H2. Multi-agent configurations consistently reduce the rare–common F_2 gap relative to single-agent baselines across the model panel (paired bootstrap on gap difference, 95% CI).

H3. Multi-Agent (Full) (M1) is statistically distinguishable from Multi-Agent (Prompt-Only) (M2) on F_2 (two-sided McNemar exact).

RQ3 is descriptive: it asks which auditability properties each configuration enables, and is therefore addressed by system-design documentation rather than statistical hypothesis test. The per-configuration trace primitives are catalogued in §4.6 and the schema-enforced grounding rates are reported in Table 5.9.

1.5 Contributions

C1 **Cross-architecture benchmark.** 5 configurations $\times$ 11 frontier models on CUAD, yielding 163,644 paired predictions analysed under McNemar exact, paired bootstrap, and BH correction (Chapter 3).

C2 **Family $\times$ tier interaction.** Pooled F_2 spans a 0.011-point band across configurations; per-family and per-tier winners differ, so no architecture is universally optimal (§5.2).

C3 **Classification-gate cascade.** Pipeline decomposition exhibits an upstream-rejection cascade: Gemini 2.5 Flash's false-negative rate rises from 9.6% under Zero-Shot to 42.2% under Classify-Extract, of which 97.9% is gate-attributable. Direction is predictable from a B0 calibration pass (§5.5).

C4 **Routing redundancy.** The LLM router agrees with a deterministic category-to-specialist mapping on 99.91% of requests, and the consolidated ablation matches it on F_2 while producing tighter spans (§5.4).

The full source code, prompt registry, experiment configurations, and run artefacts underlying every table and figure are released at `https://github.com/ndtrung00/agentic-ai-for-clm` (§4.7).

1.6 Thesis Outline

Chapter 2 defines the foundational concepts (LLMs, agentic systems, CUAD, CLM) and surveys prior multi-agent legal-AI systems along four dimensions: topology, routing, specialisation granularity, and verification structure. Chapter 3 specifies the five configurations (B0–M2), the 11-model panel, the sampling strategy, the three pre-registered hypotheses, and the statistical test suite. Chapter 4 documents the LangGraph implementation, the YAML prompt registry, the output schema, and the reproducibility infrastructure. Chapter 5 reports F_2, F_1, precision, recall, cost, and span-quality results across the 55-run grid and tests H1–H3. Chapter 6 interprets the headline null result, the classification-gate cascade, routing redundancy, and operating-point sensitivity. Chapter 7 restates the four contributions, answers the research questions, and identifies three open directions.

2 Background and Related Work

Evaluating whether multi-agent architectures improve contract clause extraction requires grounding in four technical areas: what CLM systems do and where extraction fits within them (§2.1), how LLMs generate text and why long-context tasks are structurally different (§2.2), what agentic decomposition adds and at what cost (§2.3), and what CUAD measures and why its class distribution matters (§2.4). Sections 2.5–2.8 then place the five configurations studied here against prior single-agent and multi-agent contract systems, survey evidence on when decomposition helps and when it does not, and establish the evaluation conventions inherited from ContractEval [Liu+25]. Section 2.9 consolidates the open empirical question this thesis addresses.

2.1 Contract Lifecycle Management

CLM denotes the systematic management of a contractual agreement across six canonical stages: request, author, negotiate, execute, monitor, and renew or retire [Wor25]. Each transition produces auditable artefacts (drafts, redlines, signature blocks, obligation registers), and each stage admits distinct algorithmic interventions, from clause assembly at the authoring stage to obligation extraction at the monitoring stage. Figure 2.1 visualises the lifecycle; the NLP-heavy stages are highlighted.

World Commerce & Contracting estimates that organisations forfeit $\approx 9\%$ of annual contract value through poor contract management, with detection-stage failures (missed clauses, unobserved obligations, undetected deviations) dominating the loss distribution [Wor25]. Goldman Sachs places the total addressable population of U.S. legal-services tasks exposed to generative AI at approximately 44%, with contract review repeatedly identified as the highest-exposure subset [Hat+23].

The commercial CLM market is consolidated around platforms documented in the Gartner Magic Quadrant for Contract Life Cycle Management [SMP24]. Sirion, Icertis, Ironclad, and DocuSign are the four vendors consistently designated Leaders, each having rebuilt portions of their analysis stack on LLMs since 2023. Sirion markets an Extraction Agent and a Redline Agent under an "agentic" architecture. Icertis ships ICI

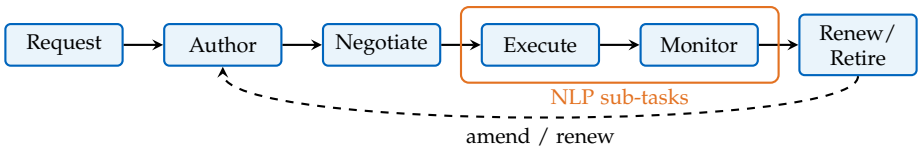


Figure 2.1: CLM lifecycle. Clause extraction, risk identification, and obligation tracking (highlighted) occur primarily in the Execute and Monitor stages.

Copilots backed by the Microsoft Azure OpenAI Service. Ironclad embeds AI Assist for redlining and Repository AI for post-signature search. DocuSign closed its US$165M acquisition of Lexion in May 2024, folding Lexion's analytics into the Intelligent Agreement Management (IAM) platform. Adjacent vendors include Spellbook (in-Word drafting on GPT-4 class models) and Harvey, whose contract-analysis workflow serves A&O Shearman and PwC [AO 23; PwC23]. Across this set, public product documentation describes single-prompt LLM calls, retrieval-augmented prompts, or task-specific fine-tunes; as of the literature review conducted for this thesis (April 2026), no Leader had publicly disclosed a deployed multi-agent architecture for clause-level analysis.

2.2 Large Language Models

An LLM is a neural language model parameterised by θ that factorises a token sequence $\mathbf{x} = (x_1, \ldots, x_n)$ autoregressively:

$$p_\theta(x_1, \ldots, x_T) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}). \tag{2.1}$$

Training minimises the negative log-likelihood $\mathcal{L}(\theta) = -\sum_t \log p_\theta(x_t \mid x_{<t})$ over large text corpora following the Transformer derivation [Dev+19]. Modern frontier LLMs extend Eq. (2.1) with three post-training techniques used in every model in the panel: supervised instruction tuning, preference alignment via reinforcement learning from human or AI feedback (RLHF), and structured-output fine-tuning that turns free-form generation into validated JSON and tool calls.

Table 2.1 summarises the frontier-model releases relevant to the experimental envelope of this thesis. Models included in the experimental evaluation are specified in §3.3; the remaining entries provide background on the frontier model releases.

A parallel layer of legal-domain products consumes these frontier models. Harvey serves A&O Shearman, PwC, O'Melveny & Myers, and Macfarlanes, and has expanded

Table 2.1: Frontier LLM releases referenced in this thesis. Context-window figures denote the standard production context; beta long-context modes noted parenthetically. Capabilities summarised from official announcements.

Model	Provider	Release	Context	Notes
GPT-4	OpenAI	Mar 2023	8K / 32K	Multimodal preview, RLHF tuned
GPT-4 Turbo	OpenAI	Nov 2023	128K	Function calling GA, JSON mode
GPT-4o	OpenAI	May 2024	128K	Native multimodal I/O
GPT-5	OpenAI	Aug 2025	400K	Unified router over fast/-thinking modes
Claude 3 Opus	Anthropic	Mar 2024	200K	Tool use API
Claude 3.5 Sonnet	Anthropic	Jun 2024	200K	Computer-use beta, arti-facts
Claude Sonnet 4	Anthropic	May 2025	200K	Extended thinking; MCP code-exec tools
Claude Sonnet 4.5	Anthropic	Sep 2025	200K (1M beta)	Agentic/tool-orchestration focus
Claude Haiku 4.5	Anthropic	Oct 2025	200K	Cost-optimised agentic tier
Claude Opus 4.5	Anthropic	Nov 2025	200K	SWE-bench Verified >80%
Gemini 1.5 Pro	Google	Feb 2024	1M	Mixture-of-experts, long-context
Gemini 2.0 Flash	Google	Dec 2024	1M	Multimodal Live API
Gemini 2.5 Pro	Google	Jun 2025	1M	Thinking budget control, MCP support
Gemini 3 Pro	Google	Nov 2025	1M	Agentic/tool-use frontier

into Vault-based document review and Agent Builder workflows [AO 23; PwC23]. Lexis+ AI and Thomson Reuters CoCounsel embed retrieval-augmented generation over proprietary legal corpora. None of these systems publishes peer-reviewed evaluations of clause-level extraction on public benchmarks as of this review. ContractEval [Liu+25] is the only large-scale academic evaluation of LLMs on contract clause extraction conducted on a public benchmark with reproducible methodology.

2.3 Agentic AI and Multi-Agent Systems

An LLM-based agent is a system in which one or more LLM calls are wrapped in a control loop that observes state, plans, invokes tools, and updates memory [Yao+23b;

Wei+22]. A Multi-Agent System (MAS) replaces a single such loop with multiple cooperating loops, each parameterised by a role, a tool surface, and (often) a different model. Five design axes locate any MAS in the design space; Figure 2.2 maps the options and marks those ablated in this thesis.

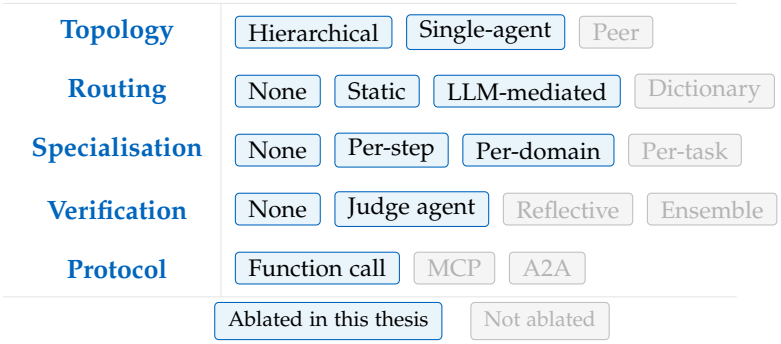


Figure 2.2: Multi-agent design space: five structural axes and their principal options. Shaded boxes mark options covered by at least one experimental configuration (B0, B1, M0, M1, M2) in this thesis. Chapter 3 specifies which configuration selects which option.

The empirical literature has converged on five design axes along which any MAS can be located. *Topology* distinguishes hierarchical configurations (orchestrator dispatches to subordinate workers) from peer configurations (agents communicate without a central coordinator), as in MetaGPT's role-pipeline [Hon+24] and CAMEL's role-playing pairs [Li+23]. *Routing* distinguishes static pipelines (fixed step order), LLM-mediated routing (a router agent selects a downstream worker), and dictionary-based routing (a deterministic feature-to-worker map). *Specialisation granularity* ranges from per-task agents (one per CUAD category) through per-domain agents (one for risk, IP, temporal) to per-step agents (extract, verify, format). *Verification structure* ranges from none, through reflective self-critique [Shi+23], multi-agent debate, judge-agent verification, tree search [Yao+23a], majority-vote ensembles such as Mixture-of-Agents [Wan+24], and self-consistency [Wan+23b]. The *communication protocol* is the lowest layer: function calling via vendor APIs, the Model Context Protocol (MCP) released by Anthropic in November 2024 [Ant24], the Agent2Agent (A2A) protocol released by Google in April 2025 and donated to the Linux Foundation in June 2025 [Sur+25], or framework-specific structured messages.

Production-relevant orchestrators include AutoGen [Wu+23], MetaGPT [Hon+24], LangGraph [Lan24a], CrewAI, and OpenAI's Agents SDK. Chapter 3 ablates the CLM-relevant slice of this design space.

2.4 The CUAD Benchmark

The Contract Understanding Atticus Dataset (CUAD) is the canonical public benchmark for clause-level contract review [Hen+21]. It consists of 510 commercial contracts annotated against 41 categories of legal clauses. The annotation pipeline employed law students who completed 70–100 hours of contract-review training under the supervision of practising attorneys [Hen+21], a provenance restated by ContractEval as a strength relative to crowd-sourced legal datasets [Liu+25]. Table 2.2 reproduces the dataset statistics.

Table 2.2: CUAD dataset statistics [Hen+21].

Property	**Value**
Contracts	510
Clause categories	41
Expert annotations	>13,000 spans
Source	EDGAR commercial filings
Train / test split	399 / 102
Task formulation	Span extraction + binary presence
License	CC BY 4.0
Long contracts (>100k chars)	14 (in test split)

The instance count is highly imbalanced: at the (contract, category) level, the natural prevalence of positive instances is $\approx 30\%$, a structural property of contract review rather than a sampling artefact. This thesis evaluates on the full 102-contract CUAD test split (character lengths 645 – 300,768; median 25,657), of which 14 contracts exceed 100,000 characters. The full split preserves the long-contract tail, where context-window pressure is highest. The short-contract regime is also retained so that the contract-length moderation analysis (§5.8) can identify the length range over which architectural decomposition is favourable. Figure 2.3 shows pooled per-category F_1 for 13 representative categories under the zero-shot baseline (B0) across the 11-model panel of this thesis (§3.3); error bars span ± 1 standard deviation across models.

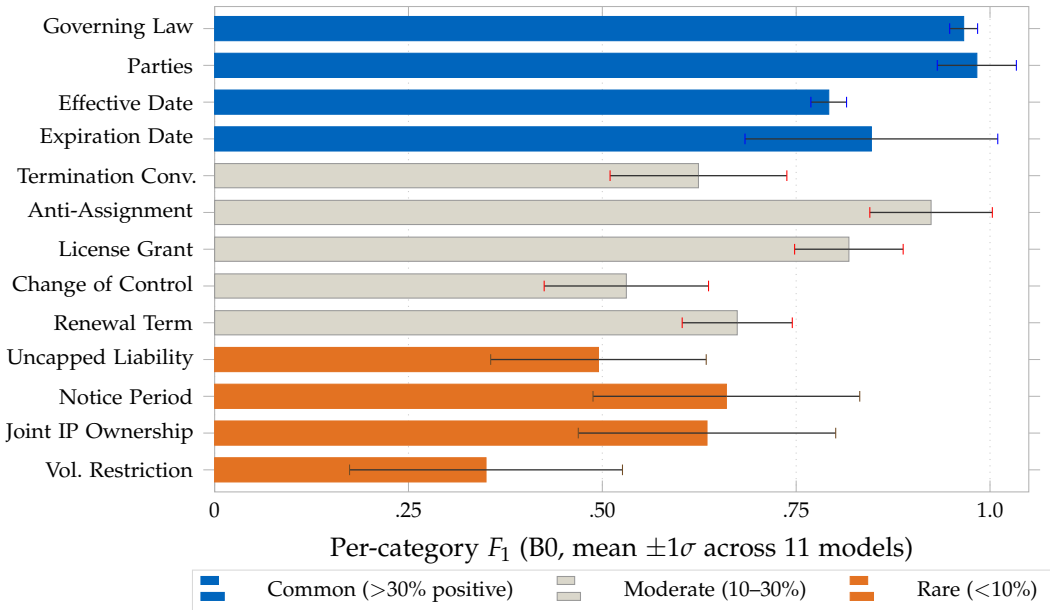


Figure 2.3: Per-category F_1 for 13 representative CUAD categories under the zero-shot baseline (B0), pooled across the 11-model panel (§3.3); error bars span $\pm 1\sigma$ across models. The rare-tier categories combine *lower mean* F_1 with *wider variance across models* (σ up to 0.17), the empirical pattern that motivates the per-domain specialist design evaluated in Chapter 3.

2.5 Contract Analysis Methods

The methodological lineage of automated contract analysis runs from rule-based extractors through statistical sequence labellers to fine-tuned and prompted Transformers. Early systems used hand-written regular expressions and gazetteers over part-of-speech (POS)-tagged text. Linear-chain Conditional Random Fields [LMP01] and BiLSTM–CRF taggers [Lam+16] formalised the task as token-level sequence labelling and dominated the LEDGAR [Tug+20] and ContractNLI (Natural Language Inference) [KM21] eras. The release of BERT [Dev+19] and legal-domain LEGAL-BERT [Cha+20] shifted the regime from feature engineering to pre-train-and-fine-tune. CUAD's reference numbers were obtained with RoBERTa-style extractive question answering [Hen+21]. MAUD [Wan+23a] extended the paradigm to merger-agreement understanding.

The LLM-prompting regime is characterised by two reproducible studies. Better Call GPT [Mar+24] reported that GPT-4 with structured prompting can match or

exceed human associates on contract-review subtasks under controlled conditions. ContractEval [Liu+25] evaluated 19 LLMs (4 proprietary, 15 open-source) on CUAD, found a persistent gap between proprietary and open-source models, and identified the named failure mode of *laziness*: the model declines to extract any span when the clause is present. ContractEval also documented a tension between reasoning mode and correctness: chain-of-thought-style decoding improved Jaccard span overlap on positive instances but degraded overall correctness, particularly on negative instances.

Table 2.3 compares the main contract-analysis benchmarks.

Table 2.3: Main contract-analysis benchmarks.

Dataset	Task	Size	Categories	In evaluation
CUAD [Hen+21]	Span extraction	510 contracts	41	✓
ContractNLI [KM21]	Doc-level NLI	607 NDAs	17	—
MAUD [Wan+23a]	Reading comp.	152 M&A agmt.	152	—
LEDGAR [Tug+20]	Multi-label cls.	60k contracts	100+	—

Why multi-agent for contract analysis. The long-contract tail of the CUAD test split exceeds 100,000 characters; explicit decomposition makes per-call context tractable where single-prompt formulations must truncate or chunk silently. The 41 CUAD categories partition into risk-oriented, IP/commercial, temporal, and consideration-oriented clusters, providing a natural basis for domain specialisation. Legal review is recall-asymmetric: a missed clause exposes the client to undisclosed obligations whereas a spurious extraction is corrected during attorney review at lower cost, motivating F_2-weighted evaluation (Section 2.8) and architectures that boost recall at controlled precision cost. Regulated environments additionally require traceable per-decision provenance: the EU AI Act treats certain legal AI uses as high-risk [EC24] and GDPR requires data-protection accountability [EC16]; multi-agent architectures with explicit verifier or judge agents produce that trace at the architectural level.

2.6 Multi-Agent Legal AI

Table 2.4 codes the published multi-agent legal systems most directly comparable to this thesis along the five design axes introduced in Section 2.3.

Table 2.4: Multi-agent legal AI systems coded against the five design-space axes of Section 2.3. All prior systems evaluate a single architecture without ablation.

System	Task	Topology	Routing	Specialisation	Verification
ChatLaw v2 [Cui+24]	Legal QA (CN)	Hierarchical	LLM-mediated	Per-domain	Self-check
PAKTON [Rap+25]	Contract review	Hierarchical	Static	Per-step (3)	None
L-MARS [WY25]	Statutory research	Hierarchical	LLM-mediated	Per-step + retrieval	Judge agent
Lawma [Dom+24]	Legal annotation	Single-agent	N/A	Per-task (fine-tune)	None
Layered CoT [San25]	Statutory reasoning	Single-agent	Static	Per-step CoT layers	Self-consistency
This thesis	CUAD test split (102)	Both	All three	All three	None (M0/M1); reflective verifier untested, §7.4

ChatLaw v2 [Cui+24] couples a Mixture-of-Experts router with role-conditioned domain agents for Chinese legal consultation; verification is a self-check pass and there is no public ablation against single-agent variants. PAKTON [Rap+25] pipelines a reading agent, an analysis agent, and a drafting agent over open-ended contract-review prompts; the system is evaluated qualitatively and not on CUAD. L-MARS [WY25] combines a planner, a retriever, and a judge agent on statutory research; performance is reported at system level without architectural decomposition. Lawma [Dom+24], while not multi-agent, provides the strongest counter-evidence to architectural complexity in the legal domain: a fine-tuned 8B Llama 3 model exceeds GPT-4 by 20.0 accuracy points on legal annotation, and the smallest variant (135M) is competitive with Claude 3.7 Sonnet. The layered chain-of-thought system of Sanwal [San25] stays within a single agent but stacks specialist prompts and self-consistency, reporting gains over flat prompting on statutory reasoning.

Across this set, every prior system reports performance for a single chosen architecture without ablating it against simpler or alternative configurations on the same evaluation

envelope. The question of whether the architecture or the model is responsible for observed performance therefore remains open. This thesis addresses it directly through cross-architecture ablation on a fixed CUAD evaluation harness.

2.7 Conditions for Multi-Agent Effectiveness

Gao, Y. Li, B. Liu, et al. [Gao+25] show on a battery of reasoning and tool-use tasks that single, well-prompted LLMs frequently match or exceed multi-agent configurations once the latter are corrected for cost and prompting parity, and that observed multi-agent gains often disappear under stronger single-agent baselines. Cemri, Pan, Yang, et al. [Cem+25] taxonomise multi-agent failure modes (specification drift between agents, inter-agent verification gaps, lossy summarisation across hand-offs) and document that these failures account for the majority of regressions in the systems they audit. Bogavelli, Subramani, and Sharma [BSS25] run a large architectural sweep across orchestration patterns and base models and find substantial model-family $\times$ architecture interactions: an architecture that is optimal under one model family is often dominated under another. Sanwal [San25] demonstrates that layered single-agent chain-of-thought can recover much of the benefit attributed to multi-agent decomposition without paying the coordination cost.

These findings bear directly on the three working hypotheses. (H1) A generic claim of multi-agent superiority on CUAD clause extraction is unlikely to survive a properly normalised single-agent baseline [Gao+25], so the hypothesis is stated and evaluated as conditional. (H2) The relative ordering of architectures will depend on the underlying model family [BSS25], encoded as an explicit interaction term in the experimental design. (H3) Classification-gate cascades (a router agent that decides whether to extract before invoking an extractor) are a natural counter to the verification-gap and laziness failures catalogued in [Cem+25], motivating the M0 ablation in Chapter 3. Whether this counter is effective is tested empirically in §5.5.

2.8 Evaluation Methodology

This section consolidates the evaluation primitives used throughout the thesis. Chapter 3 references these definitions without restating them.

Span and instance metrics. For a category c on a contract d, let $S^*_{c,d}$ be the set of gold spans and $\hat{S}_{c,d}$ the set of predicted spans. A predicted span $\hat{s}$ matches a gold span s when their character-level Jaccard overlap exceeds threshold τ=0.5 following [Hen+21]:

$$J(\hat{s}, s) = \frac{|\hat{s} \cap s|}{|\hat{s} \cup s|}. \tag{2.2}$$

At the (contract, category) level, the prediction is true-positive if any predicted span matches any gold span at threshold τ, false-negative if gold spans exist with no match, false-positive if predicted spans exist with no match, and true-negative if both sets are empty. Precision P and recall R aggregate over the natural class distribution.

F_β**.** Recall is privileged over precision in legal review by a factor commonly fixed at β=2:

$$F_\beta = \frac{(1+\beta^2)\, P\, R}{\beta^2 P + R}, \qquad \beta = 2. \tag{2.3}$$

F_1 is reported alongside F_2 for compatibility with prior CUAD work. Instance-level F_β at the (contract, category) granularity is the primary metric reported in Chapter 4, as it aligns with the deployment unit (a reviewer's decision per category per contract).

Failure-mode metrics. ContractEval [Liu+25] introduced *laziness* as the per-model rate at which an LLM declines to extract on instances where a gold span exists:

$$\lambda = \frac{|\{(c,d) : |\hat{S}_{c,d}| = 0 \ \wedge \ |S^*_{c,d}| > 0\}|}{|\{(c,d) : |S^*_{c,d}| > 0\}|}. \tag{2.4}$$

The dual failure mode is *over-extraction*: the per-model rate of producing spans on negative instances:

$$\omega = \frac{|\{(c,d) : |\hat{S}_{c,d}| > 0 \ \wedge \ |S^*_{c,d}| = 0\}|}{|\{(c,d) : |S^*_{c,d}| = 0\}|}. \tag{2.5}$$

Equations (2.4) and (2.5) are a coupled pair: evaluations that resample to a balanced class distribution suppress Eq. (2.5) and hide the precision cost of recall-boosting interventions. The evaluation therefore uses exclusively the natural CUAD class distribution and reports both metrics.

Statistical testing. Every architecture comparison is paired on the same contract set; McNemar's test handles instance-level binary outcomes [McN47] and the paired bootstrap with B=300 resamples handles span-level continuous metrics [Efr79; RM05]. Multiple-comparison control across the architecture grid uses Benjamini–Hochberg false-discovery-rate correction at q=0.05 [BH95]. Sample-size and power considerations follow Dror, Baumer, Shlomov, and Reichart [Dro+18] and Card, Henderson, Khandelwal, et al. [Car+20]; the 102-contract test split gives $\geq$ 0.8 power to detect absolute F_2 differences of $\approx$2.5 percentage points at the instance level under the bootstrap.

Sampling design. CUAD's natural clause-presence rate is $\approx$ 30% positive at the instance level. Resampling to balance positive and negative instances inflates apparent precision and suppresses Eq. (2.5), detaching the evaluation from any deployable distribution where 41 categories are scanned per contract and most are absent. The present evaluation retains every positive (contract, category) pair and downsamples negatives to a measured 41.5% positive rate (§3.2); this preserves rare-tier statistical power while keeping over-extraction (Eq. (2.5)) observable [Liu+25].

ContractEval as direct empirical predecessor. ContractEval [Liu+25] evaluates 19 LLMs (4 proprietary, 15 open-source) on 4,128 (contract, category) samples across the full 41-category CUAD schema, holding prompt and decoding constants across models, and reporting Jaccard span overlap, instance-level correctness, and laziness. Its principal findings are: a persistent proprietary–open-source gap on legal extraction; a non-monotonic effect of explicit reasoning (chain-of-thought improves Jaccard on positive instances while degrading correctness through inflated false positives); and laziness as a persistent, model-dependent and category-dependent failure mode. Three properties of the ContractEval design define the gaps the present thesis closes: the architectural axis is held fixed at single-prompt single-model; model-family interactions are not analysed; and over-extraction (Eq. (2.5)) is not tracked as a first-class metric. The present thesis retains ContractEval's instance-level scoring, laziness reporting, and τ=0.5 Jaccard threshold; departs on class distribution (upsampled to 41.5% positive, §3.2); and adds over-extraction (Eq. (2.5)) and a model-family interaction term as first-class evaluation axes.

Prompt sensitivity as a validity threat. The prompt-sensitivity index of Chatterjee, Renduchintala, Bhatia, and Chakraborty [Cha+24] quantifies the sensitivity of LLM outputs to semantically equivalent prompt rewrites and reports non-trivial sensitivity even on well-defined tasks. Prompt sensitivity is therefore a first-order validity threat

for any architecture comparison in which architectures differ in the number, role, or wording of agent prompts. Chapter 3 controls this threat by holding system-prompt scaffolds fixed across architectures where the role permits, and reports an analogous sensitivity check on the architectural ablations. Observability and trace inspection use Langfuse [Lan24b]; orchestration uses LangGraph [Lan24a].

2.9 Research Gap

(i) No published study performs a controlled cross-architecture ablation on CUAD-class clause extraction; the closest predecessor [Liu+25] varies model identity only. (ii) No published study reports a model-family $\times$ architecture interaction analysis on this task, despite the prediction in [BSS25] that such interactions dominate single-axis effects. (iii) No published study tracks both Eq. (2.4) and Eq. (2.5) under any deployable class distribution that preserves over-extraction as observable, leaving the recall–precision asymmetry intrinsic to legal review unobserved. (iv) Production CLM platforms (Sirion, Icertis, Ironclad, DocuSign) disclose single-prompt or retrieval-augmented LLM deployments with no public multi-agent ablation, leaving the architectural design space untested in the regime where the question is commercially decisive.

This thesis designs and evaluates a slice of the multi-agent design space (topology $\times$ routing $\times$ specialisation $\times$ verification) on the 102-contract CUAD test split under the ContractEval protocol extended with Eq. (2.5) and a model-family interaction term, in order to determine, with statistical control across the architecture grid [BH95; Dro+18], under which conditions multi-agent decomposition is justified for CLM deployment and under which conditions it is dominated by a stronger single-agent baseline.

3 Methodology

Each architectural comparison holds one axis constant while varying one: B1 vs. B0 isolates chain-of-thought reasoning, M0 vs. B0 isolates the classification gate, and M1 vs. M2 isolates routing from prompt content. This controlled design allows performance differences to be attributed to architecture rather than model or data variation across $5 \times 11 = 55$ runs on the 102-contract CUAD test split.

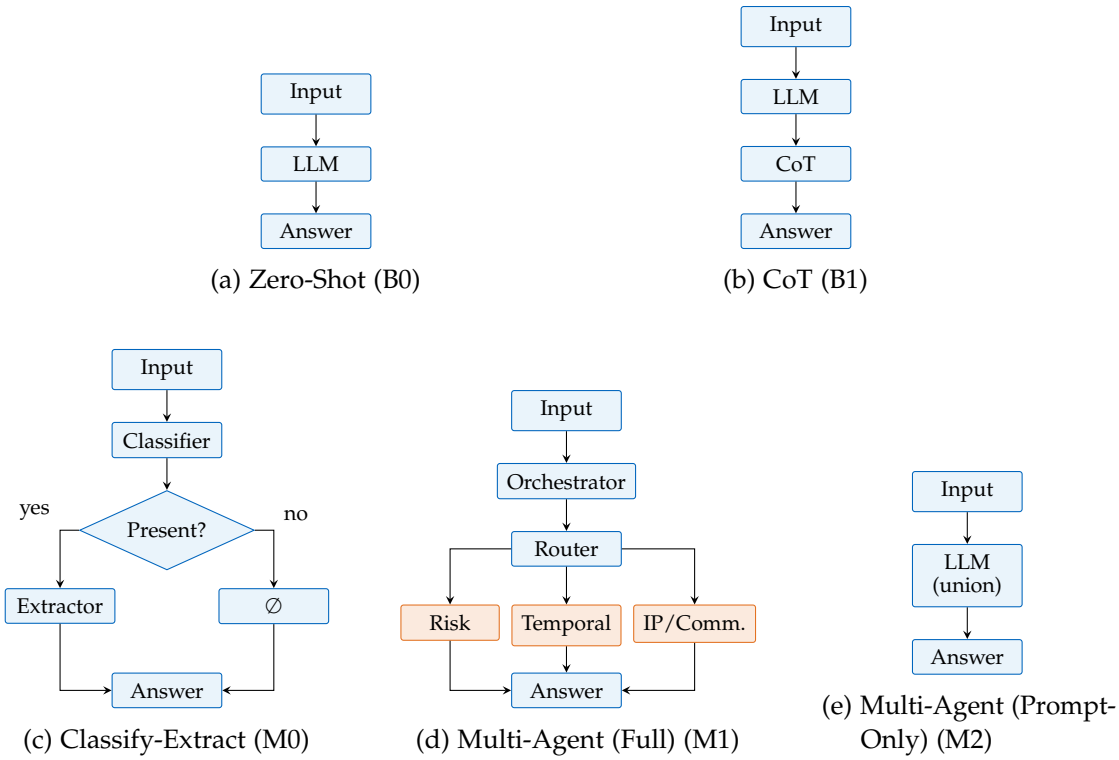
3.1 Research Design

Five configurations are evaluated against 11 frontier LLMs on the 102 contracts of the CUAD test split (character lengths 645 – 300,768; median 25,657), yielding $5 \times 11 = 55$ official runs and 163,644 analysed predictions. Differences between configurations are minimised to a single architectural axis per comparison (Table 3.1), following the controlled comparative protocol of [Dro+18; Car+20]. Every statistical test is paired on the intersection of sample IDs, so performance differences are attributable to architecture or model, not dataset drift.

Table 3.1: Five evaluated configurations. B1 vs. B0 isolates chain-of-thought; M0 vs. B0 isolates the classification gate; M1 vs. M2 isolates routing and specialist decomposition (architecture ablation). †Artifact IDs used in the experiment repository: B0 = B1, B1 = B4, M0 = M0, M1 = M1, M2 = M6.

ID	Name	Stages	Routing	Specialisation
B0	Zero-Shot	1	None	Generalist
B1	CoT	1	None	Generalist + chain-of-thought
M0	Classify-Extract	2	Static gate	Generalist
M1	Multi-Agent (Full)	2	LLM router	Three domain specialists
M2	Multi-Agent (Prompt-Only)	1	None	Union of M1 specialist prompts

Figure 3.1 shows the message-flow diagram for each configuration.



(a) Zero-Shot (B0) (b) CoT (B1) (c) Classify-Extract (M0) (d) Multi-Agent (Full) (M1) (e) Multi-Agent (Prompt-Only) (M2)

Figure 3.1: Message-flow diagrams for the five configurations. Input = contract text + clause-category indicator. Blue nodes: generalist LLM invocations. Orange nodes: domain-specialist prompts. ∅ = silent no-clause return.

3.2 Dataset and Sampling

CUAD [Hen+21] is the evaluation corpus (see Table 2.2 for statistics). The full 102-contract CUAD test split is used; contract lengths range from 645 to 300,768 characters (median 25,657), spanning short NDAs to long master agreements. The 102-contract corpus yields $102 \times 41 = 4{,}182$ raw (contract, category) pairs per evaluation pass, of which $\approx$2,999 enter each run after the sampler described below.

Sampling strategy. A `neg_ratio=0.7` sampler retains every positive (contract, category) pair and approximately 60% of the negative pairs, yielding $\approx$2,999 samples per run with a measured positive rate of 41.5%. The sampler upsamples the positive class relative to CUAD's natural $\approx$30% prior to preserve statistical power on rare-tier

categories, where positive instances are sparse; without upsampling, fewer than ten positive instances per rare category would remain. Aggressive resampling toward a balanced class distribution is rejected because it suppresses the false-positive denominator and masks over-extraction (Eq. 2.5); the chosen 41.5% rate keeps both laziness (Eq. 2.4) and over-extraction observable. Sampling is seeded (`seed=42`) so the same (contract, category) pairs are evaluated across all configurations, making every statistical test paired at the instance level. Each run yields ≈2,999 predictions; 1,297 of 164,941 total (0.79%) were excluded for malformed classification tokens.

Prevalence tiers. The 41 CUAD categories are partitioned into three tiers (*common*, *moderate*, *rare*) by natural positive rate in the 102-contract test split. Tier cutpoints are fixed before any model is run.

3.3 Model Selection

Table 3.2 lists the 11 frontier proprietary models in the panel. Three families span the providers used in commercial CLM (Anthropic, Google, OpenAI), so model-family effects predicted by [BSS25] can be observed; within each family the panel spans a compute-tier range from light (Haiku, Flash, Nano) to heavy (Opus, Pro), enabling capability × architecture interactions to be observed. Model identifiers are pinned to specific snapshots (see §4.2) for reproducibility.

Inference parameters. All model calls used `temperature=0` for deterministic output and `max_tokens=4096`. Official runs were executed between **14 April 2026** and **17 April 2026**. Snapshot IDs are listed in Table 3.2; the three Google and two OpenAI entries use unversioned aliases that resolve to the provider's current release at call time.[1]

Open-weights models are excluded from the main panel on two structural grounds: the commercial CLM platforms surveyed in Section 2.1 deploy proprietary APIs, restricting

[1]Strict reproducibility requires pinned dated snapshots. OpenAI publishes dated aliases (e.g. `gpt-4.1-2025-04-14`); Anthropic publishes `claude-opus-4-6-<date>`. Pinned variants were not available for all models at the time experiments were run. As a consequence, a silent provider-side release of any aliased model between 14 and 17 April 2026 would have shifted that model's behaviour mid-experiment in a way the run logs cannot detect; no post-hoc API-fingerprint check (e.g. deterministic-prompt response stability) was performed, and the family-by-configuration interaction observed in Chapter 5 for the affected models (Gemini 2.5/3/3.1 and GPT-4.1/Mini/Nano) cannot be ruled out as a snapshot-drift artefact rather than a stable model property.

Table 3.2: Eleven frontier proprietary models evaluated. Three families; compute-tier range covers light-to-heavy within each family. Snapshot IDs are the exact provider model strings used in all API calls.

Family	Model	Tier	Snapshot ID
Anthropic	Claude Haiku 4.5	Light	`claude-haiku-4-5-20251001`
	Claude Sonnet 4	Mid	`claude-sonnet-4-20250514`
	Claude Sonnet 4.6	Mid	`claude-sonnet-4-6-20251001`
	Claude Opus 4.6	Heavy	`claude-opus-4-6`
Google	Gemini 2.5 Flash	Light	`gemini-2.5-flash`
	Gemini 2.5 Pro	Heavy	`gemini-2.5-pro`
	Gemini 3 Flash	Light	`gemini-3-flash-preview`
	Gemini 3.1 Pro	Heavy	`gemini-3.1-pro-preview`
OpenAI	GPT-4.1	Mid	`gpt-4.1`
	GPT-4.1 Mini	Light	`gpt-4.1-mini`
	GPT-4.1 Nano	Light	`gpt-4.1-nano`

external-validity claims to that deployment population; and the long-contract tail of the corpus requires context windows of $\geq$ 128K tokens, which most open-weights models do not support. ContractEval's 19-model evaluation [Liu+25] documents a persistent proprietary-vs-open-source gap on extraction correctness; pilot work (not formally reported) suggested this gap persists under multi-agent decomposition, though that experiment should be understood as a scope decision rather than an empirical claim. Open-weights extension is identified as future work in Chapter 7.

3.4 Metrics

All metrics are formally defined in Section 2.8 (Equations 2.3–2.5).

F_2 **(primary).** β=2 weights recall 4× over precision. The asymmetry is justified by the deployment cost structure of CLM: a missed clause passes undetected into a signed contract, whereas a spurious extraction is caught by a reviewing lawyer. F_2 is the primary hypothesis-testing metric and the basis for H1–H3.

F_1 **(secondary).** Equal-weight F-score, reported for comparability with prior work. ContractEval reports F_1 as its headline metric [Liu+25].

Jaccard similarity (span quality). $|A \cap B|/|A \cup B|$ over token sets, computed conditional on a non-empty prediction. Because the F-score TP criterion requires full span coverage, partial extractions that cover most of a clause register as false negatives; Jaccard gives partial credit and diagnoses near-miss behaviour independently of the binary correctness signal.

Laziness rate λ (diagnostic). Fraction of positive-label instances on which the model returns "no related clause". The dominant failure mode of single-agent LLMs on rare CUAD categories is silent abstention rather than wrong extraction; λ isolates this directly.

Over-extraction rate ω (diagnostic). Fraction of negative-label instances on which the model returns a non-empty span. Complements λ: together they decompose the false-negative and false-positive error budgets without conflating them into a single F-score.

All metrics are reported pooled and per prevalence tier.

3.5 Hypotheses

H1. For every $C \in \{\text{M0, M1}\}$, C outperforms $B^\star = \text{B0}$ on F_2 across the eleven-model panel. Null: at least one C does not outperform B0 on the panel. Decision rule: reject the null *in favour of H1* only if, *for both* $C \in \{\text{M0, M1}\}$, a majority of the 11 per-model C-vs-B0 McNemar exact contrasts is BH-significant in favour [BH95] at α=0.05 within the combined m=22-test family. Per-model BH-significant positive contrasts that do not satisfy the per-config majority condition are reported as *conditional* improvements rather than as evidence for H1 itself. Multi-Agent (Prompt-Only) (M2) is excluded from the H1 family because it is the architecture-control ablation for H3 (§3.5), not a candidate alternative architecture; CoT (B1) is the chain-of-thought baseline against which the F_2 comparisons are reported descriptively in Chapter 5 but not used as the H1 reference baseline (B0 is the primary baseline per ContractEval [Liu+25]).

H2. $\Delta\mathrm{Gap}(C_{\mathrm{multi}}, B^{\star}) < 0$ where $\mathrm{Gap}(C) = F_2^{\mathrm{common}}(C) - F_2^{\mathrm{rare}}(C)$ holds across the eleven-model panel. Null: H2 does not hold on the panel. Decision rule: reject the null in favour of H2 for a given (model, configuration) pair if the upper bound of the 95% paired bootstrap CI on ΔGap is strictly less than zero. H2 is supported pooled if *a majority* of the 22 (model, configuration) contrasts satisfy this rule; otherwise H2 is rejected.

H3. Multi-Agent (Full) (M1) and Multi-Agent (Prompt-Only) (M2) are statistically distinguishable on F_2. Null: $\Delta F_2(\mathrm{M1}, \mathrm{M2}) = 0$. Decision rule: reject the null (two-sided) if the McNemar exact p-value is below α=0.05 after BH correction (m=11, one M1 vs. M2 contrast per model). The hypothesis is two-sided because direction is not pre-specified; if rejected, the direction of the effect is reported separately because direction may be inconsistent across the model panel.

3.6 Statistical Methodology

McNemar's exact test. For configurations A and B evaluated on the same instances, let $b = |\{i : A_i \text{ correct}, B_i \text{ wrong}\}|$ and $c = |\{i : A_i \text{ wrong}, B_i \text{ correct}\}|$. The two-sided exact p-value is:

$$p = 2\,\mathbb{P}\big[\mathrm{Bin}\big(b{+}c, \tfrac{1}{2}\big) \le \min(b, c)\big] \tag{3.1}$$

implemented via `scipy.stats.binomtest` [Vir+20]. The exact form is preferred over the χ^2 approximation because discordant counts are small on rare-tier subsets [McN47].

Paired bootstrap. For continuous metrics (F_2, F_1, Jaccard, gap differences), Algorithm 1 gives the paired bootstrap CI [Efr79; RM05]. We use B=300 resamples and seed the bootstrap RNG independently of the sampler with `numpy.random.default_rng(42)` so the data partition seed and the bootstrap resample seed are both pinned to the same numerical value but instantiated as separate generator objects (the bootstrap percentile CIs in Table 5.4 are bit-reproducible from the artefact bundle under this convention). The percentile method's robustness to small B on bounded statistics is documented as a sensitivity threat in Section 3.7.

Algorithm 1 Paired bootstrap confidence interval for ΔF_2

Require: Predictions $\hat{y}^A$, $\hat{y}^B$, labels y, B=300, α
1: **for** $b = 1$ **to** B **do**
2: $\mathcal{I} \leftarrow$ sample n indices with replacement from $\{1, \ldots, n\}$
3: $\delta_b \leftarrow F_2(\hat{y}^A_{\mathcal{I}}, y_{\mathcal{I}}) - F_2(\hat{y}^B_{\mathcal{I}}, y_{\mathcal{I}})$
4: **end for**
5: **return** $[\text{percentile}(\delta, \alpha/2),\ \text{percentile}(\delta, 1-\alpha/2)]$

Multiple-comparison correction. Within each hypothesis family, Benjamini–Hochberg [BH95] controls the false discovery rate (FDR) at α=0.05: reject $H_0^{(i)}$ if

$$p_{(i)} \leq \frac{i}{m}\alpha, \quad p_{(1)} \leq \cdots \leq p_{(m)}. \tag{3.2}$$

The H1 family has m=22 tests (2 multi-agent configurations $\times$ 11 models, with B0 as reference baseline); the H3 family has m=11 tests (one M1 vs. M2 contrast per model). Raw and BH-corrected p-values are both reported.

Effect size. For paired binary comparisons (McNemar's), we report Cohen's $g = b/(b+c) - 0.5$, the deviation of the discordant proportion from chance [Coh88]. For continuous metrics, we report absolute ΔF_2 with the paired bootstrap 95% CI. We follow Cohen's interpretive thresholds ($g \geq 0.05$ small, ≥ 0.15 medium, ≥ 0.25 large; $|\Delta F_2| \geq 0.02$ small, ≥ 0.05 medium, ≥ 0.08 large) [Coh88], and report effect size alongside p-values in accordance with the recommendation that the magnitude of difference is the primary research output, not statistical significance per se [SF12].

Power. Card, Henderson, Khandelwal, et al. [Car+20] caution that NLP comparisons are frequently under-powered. The pairwise sample size after intersection is $\approx$2,900–3,000 per (configuration, model) pair, of which the discordant-pair count $b+c$ on M0-vs-B0 contrasts ranges from 236 (*Claude Opus 4.6*) to 1,030 (*Gemini 2.5 Flash*) across the 11 models, with median $b+c \approx 551$. Under McNemar's exact two-sided test at α=0.05 and target power 0.80, the median discordant-pair count corresponds to a minimum detectable proportion of $p_A = b/(b+c) \geq 0.56$ (equivalently $(b-c)/(b+c) \geq 0.12$, computed via the binomial-distribution power formula [Con87]). Mapping discordant proportion to F_2 on the dominant pooled-precision regime of the panel yields a minimum detectable $|\Delta F_2|$ of approximately 0.025. Effects below this threshold are treated as inconclusive rather than as evidence of equivalence; the headline pooled spread of 0.011 on F_2 across the five configurations sits within the inconclusive band by design.

Paired comparison protocol. Intersection of sample IDs is taken symmetrically when partial runs differ in coverage; intersection size is reported with each test.

3.7 Threats to Validity

Internal. Configuration parity is maintained by holding the CUAD loader, sampling seed, prompt registry, and trace schema constant across configurations. The M1 vs. M2 contrast is the most architecturally informative but not a clean single-axis ablation: M1 differs from M2 on three jointly-varied components: prompt-format split (one focused specialist preamble vs. the textual union of three), dispatch overhead (router LLM call plus orchestrator hop), and per-call latency. Effects observed on the M1 vs. M2 contrast are therefore attributable to the joint architectural delta, not to any one component in isolation; a within-M1 ablation that disables the router decision while retaining the topology is left to future work (§7.4).

External. Single language (English), single jurisdiction (predominantly U.S. commercial contracts), single corpus (CUAD). Findings on the 102-contract test split may not extrapolate to non-CUAD clause taxonomies; the contract-length moderation analysis (§5.8) shows that the architectural ranking itself shifts across length quartiles within the corpus. Open-weights models are excluded; inference does not generalise beyond the 11 proprietary models studied. Model snapshots are pinned; provider updates can shift behaviour.

Construct. F_2 is a proxy for legal-review utility, not a direct measurement. Reviewer cost of inspecting a spurious span is workflow-dependent; this is mitigated by reporting precision, recall, λ, and ω separately so that a reviewer with a different cost ratio can re-rank architectures.

Conclusion. The prompt-sensitivity threat is established in §2.8. We mitigate it by holding prompt templates fixed across models per configuration, documenting them in full in Chapter 4, and treating prompt-perturbation analysis as future work.

Statistical. Bootstrap percentile CIs under-cover for small samples and skewed statistics [Efr79]; p-values are treated as decisive only when corroborated by the McNemar exact test. The bootstrap resample count (B=300) is below the 1,000 conservative recommendation [Dro+18]; a sensitivity check on the M0-vs-B0 contrasts for the four-model Anthropic family (*Claude Haiku 4.5*, *Sonnet 4*, *Sonnet 4.6*, *Opus 4.6*; chosen because all four runs are complete and the contrast spans the panel's capability range from light to heavy) compared the percentile CI on ΔF_2 at B=300 versus B=1,000 under the same bootstrap seed (`seed=42`, `numpy.random.default_rng`). The maximum endpoint shift was 0.0026 on the lower bound and 0.0013 on the upper bound, well within the ± 0.005 tolerance the headline reporting precision requires. The replication script is released alongside the experiment artefacts (§4.7). Model identity is a fixed factor; inference does not generalise beyond the 11 models studied.

4 System Design and Implementation

All five configurations share a common parser, sampler, and metrics layer; only the LangGraph topology and prompt registry entries differ. This design isolates architectural variation and enables the controlled comparisons of Chapter 3.

4.1 System Components and Data Flow

Figure 4.1 shows the static component view. A run is initiated by the experiment driver, which loads CUAD samples through the deterministic loader, resolves prompts through the registry, and dispatches each sample to one of the five LangGraph state machines via a unified model client. Every model call is mirrored to the trace store; final answers are persisted as JSONL (JSON Lines: one JSON object per line) records consumed by the statistical pipeline and the inspection dashboard.

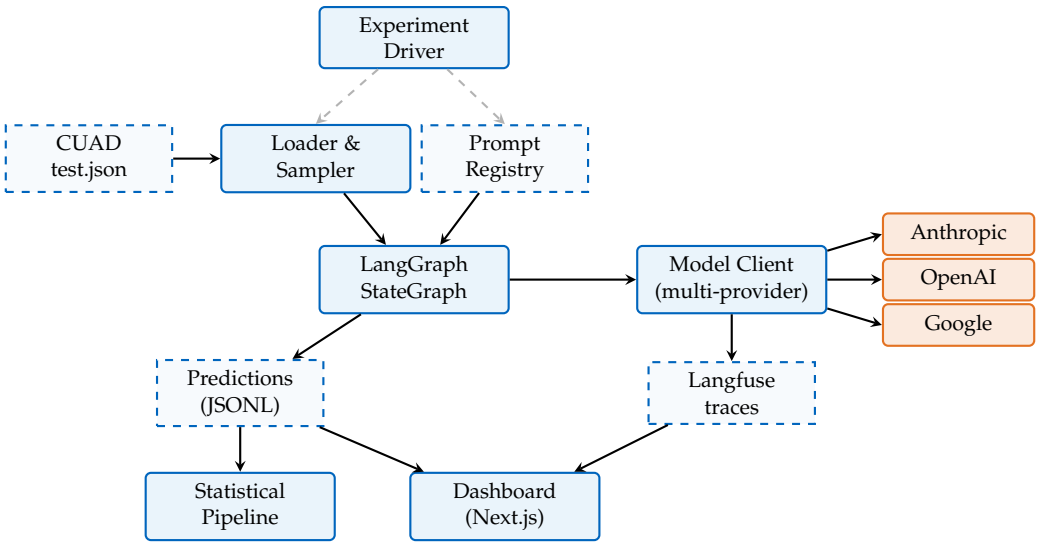


Figure 4.1: System component diagram. *Solid-border boxes*: code components (Loader, StateGraph, ModelClient, Statistical Pipeline, Dashboard). *Dashed boxes*: data stores. *Orange boxes*: external provider APIs. *Dashed arrows*: control flow (Driver initialises inputs before each run). *Solid arrows*: data flow. Only the StateGraph topology varies across B0–M2; all other components are configuration-agnostic.

4.2 Implementation Stack

The codebase is Python 3.11, managed with `uv`. Orchestration uses LangGraph [Lan24a]; observability uses Langfuse [Lan24b]; the statistical pipeline uses `pandas` [The20], `numpy` [Har+20], and `scipy` [Vir+20]. Provider SDKs are wrapped in a unified `ModelClient` that records, per call, prompt tokens, completion tokens, cache-read tokens (where supported), wall-clock latency, and provider-billed cost. The wrapper supports 19 model identifiers; only the 11 listed in Table 3.2 are exercised in official runs. The remaining 8 identifiers cover open-weights models excluded per Section 3.3 and two non-CUAD test configurations not evaluated in this study.

Inference parameters. All official model calls used `temperature=0` and `max_tokens=4096`. Official runs were executed between **14 April 2026** and **17 April 2026**. Provider snapshot identifiers for each of the 11 models are listed in Table 3.2 of Chapter 3.

4.3 Shared Abstractions

Graph state. Every configuration is a LangGraph `StateGraph` over a shared `GraphState` object (Listing 4.1). Using a shared schema enforces configuration parity: no configuration has access to fields another lacks, and the JSONL persistence format is identical across all five configurations.

Listing 4.1: GraphState schema (abridged from `src/agents/state.py`).

```
class GraphState(TypedDict, total=False):
    # Inputs (unchanged throughout graph)
    contract_text: str
    category: str # one of 41 CUAD category names
    question: str # CUAD annotation-guideline question text

    # Routing (set by orchestrator node --- M1 only)
    specialist_name: str # "risk_liability" | "temporal_renewal" | "ip_commercial"

    # Extraction (set by specialist or baseline node)
    extraction_result: ExtractionResult | None

    # Validation (set by validation node)
    validated: bool
    validation_notes: str

    # Final output
    final_result: ExtractionResult | None

    # Trace: append-reducer accumulates entries across all nodes
    trace: Annotated[list[dict[str, Any]], append_trace]

    error: str | None
```

Prompt registry. Prompts are stored as YAML and loaded once per run; the schema version hash is written to the run metadata so prompt drift is detectable post-hoc. Two roots: `prompts/baselines/` (B0, B1, and the M0 classifier and extractor) and `prompts/specialists/` (the three M1 domain specialists and the M2 consolidated prompt). A shared `system/` directory holds anti-laziness instructions and the verbatim-span output contract, referenced by all configurations, ensuring that the anti-laziness and verbatim-span output instructions are identical across B0, B1, M0, M1, and M2.

Common parser. A single parser converts every model completion to a canonical `(spans: list[str] | None)` representation. For B0, the parser accepts verbatim quoted spans or the literal string "no related clause". For B1 and M0-extractor, the parser additionally requires a `<answer>...</answer>` tag and issues a single repair re-prompt if the tag is absent. On persistent failure the record is marked `parser_status="rejected"` and excluded from official statistics (1,297 of 164,941 predictions, 0.79%).

4.4 Baseline Architectures

B0: Zero-shot single agent. B0 replicates the zero-shot prompt protocol of [Liu+25]. The model receives one user message containing the contract text, the target category name, the category indicator, and an explicit instruction to return verbatim span(s) or the literal string "no related clause". No examples, no chain-of-thought directives.

B0: zero-shot user template

You are reviewing a commercial contract. Determine whether the contract contains any clause(s) of the category below. If yes, return the verbatim span(s); each span must appear exactly as in the contract. If no, return exactly: "no related clause".
Category: *{{category}}* ***Description:*** *{{category_indicator}}*
Contract: *{{contract_text}}*

B1: Chain-of-thought single agent. B1 retains B0's task framing but elicits explicit step-by-step reasoning before a final structured answer. The model is instructed to restate what the category covers, scan for candidate spans, weigh ambiguous matches against the indicator, and emit a final `<answer>...</answer>` block. A parsing bug in pilot work caused contract passages quoted inside the chain-of-thought to be harvested as answers, inflating ω (Eq. 2.5); the fix enforces the answer tag via the common parser of Section 4.3. The fix was deployed in commit `1cf08536` on **14 April 2026**; only post-fix runs enter official statistics.

4.5 Multi-Agent Architectures

M0: Classify-then-extract. M0 decomposes each (contract, category) task into two sequential LLM calls. The *classifier* returns a single yes/no token; the *extractor* is invoked only if the classifier returns yes. The two agents share the category indicator

but have disjoint prompts: the classifier is forbidden to emit spans, the extractor assumes presence. Listing 4.2 excerpts both prompts; Listing 4.3 shows the LangGraph construction. Non-binary classifier responses (e.g. capitalised "Yes", punctuated "no.", or free-form text) are normalised by case-folding and stripping punctuation before matching; responses that do not resolve to "yes" or "no" after normalisation are treated as "no" and the record is flagged `parser_status="rejected"`, contributing to the 1,297 excluded predictions (0.79%).

Listing 4.2: M0 classifier and extractor prompts (abridged).

```
# --- classifier ---
Decide whether the contract contains any clause of the category below.
Return exactly one token: "yes" or "no". Do not return spans.
Category: {{category}} Description: {{category_indicator}}
Contract: {{contract_text}}

# --- extractor (invoked only when classifier returned "yes") ---
The contract contains at least one clause of the category below.
Return the verbatim span(s) wrapped in <answer>...</answer>.
```

Listing 4.3: M0 LangGraph construction.

```
g = StateGraph(GraphState)
g.add_node("classify", classify_node)
g.add_node("extract", extract_node)
g.set_entry_point("classify")
g.add_conditional_edges(
    "classify",
    lambda s: "extract" if s["classifier_verdict"] == "yes" else END,
    {"extract": "extract", END: END},
)
g.add_edge("extract", END)
```

M1: Orchestrator with three specialists. M1 routes each request to one of three domain specialists via an LLM router. The category-to-specialist assignment is documented in Table 4.1; it is fixed before any model is run and is committed as `CATEGORY_ROUTING` in `src/agents/orchestrator.py`.

The router system prompt is reproduced verbatim below from `ROUTING_SYSTEM_PROMPT` in the artefact. Routing accuracy across the 11-model panel is reported in Section 5.4.

Table 4.1: Category-to-specialist assignment used by M1. All 41 CUAD categories are listed; the mapping is fixed before any model is run.

Risk/Liability (13)	**Temporal/Renewal (11)**	**IP/Commercial (17)**
Cap on Liability	Document Name	IP Ownership Assignment
Uncapped Liability	Parties	Joint IP Ownership
Liquidated Damages	Agreement Date	License Grant
Insurance	Effective Date	Non-Transferable License
Warranty Duration	Expiration Date	Affiliate License-Licensor
Audit Rights	Renewal Term	Affiliate License-Licensee
Non-Disparagement	Notice Period to Terminate Renewal	Unlimited License
Covenant Not to Sue	Termination for Convenience	Irrevocable or Perpetual License
Third Party Beneficiary	Anti-Assignment	Source Code Escrow
Most Favored Nation	ROFR/ROFO/ROFN	Exclusivity
Change of Control	Governing Law	Non-Compete
Post-Termination Services		No-Solicit of Customers
Minimum Commitment		No-Solicit of Employees
		Competitive Restriction Exception
		Revenue/Profit Sharing
		Price Restrictions
		Volume Restriction

M1 router system prompt (verbatim from `src/agents/orchestrator.py`)

You are a routing agent for a contract analysis system. Your job is to read a question about a contract clause and decide which specialist should handle it.
risk_liability *— Expert in risk allocation, liability, and protective provisions. Covers: liability caps and uncapped liability, liquidated damages, insurance requirements, warranty duration, audit rights, non-disparagement, covenants not to sue, third-party beneficiaries, most favored nation clauses, change of control provisions, post-termination services, and minimum commitments.*
temporal_renewal *— Expert in temporal aspects, document identification, and contract lifecycle. Covers: document name and party identification, agreement/effective/expiration dates, renewal terms and notice periods for renewal termination, termination for convenience, anti-assignment provisions, rights of first refusal/offer/negotiation, and governing law.*
ip_commercial *— Expert in intellectual property rights and commercial restrictions. Covers: IP ownership and assignment, joint IP ownership, license grants (transferability, affiliate licenses, unlimited/perpetual licenses), source code escrow, exclusivity, non-compete and non-solicitation clauses, competitive restriction exceptions, revenue/profit sharing, price restrictions, and volume restrictions.*
Respond with ONLY a JSON object: {"specialist": "<name>", "reasoning": "<brief explanation>"}

Each specialist's prompt is a focused reading instruction for its cluster. The Risk/Liability specialist is shown below; Temporal and IP/Commercial follow the same template.

M1 Risk/Liability specialist system prompt (abridged)

You are a contract analyst specialising in risk-allocation provisions: liability caps, uncapped exclusions, indemnification, insurance, warranty, audit rights, non-disparagement, third-party beneficiaries, most-favored-nation, change of control, post-termination services, and minimum commitments. For these categories, recall of monetary thresholds and explicit carve-outs ("except for...", "not in excess of...") is critical and preferred over summary paraphrase.
Return the verbatim span(s) of any clause of the category below, or "no related clause" if none is present.
Category: *{{category}}* ***Description:*** *{{category_indicator}}*
Contract: *{{contract_text}}*

Figure 4.2 traces a single M1 invocation, showing the parent–child call structure reproduced in the Langfuse trace.

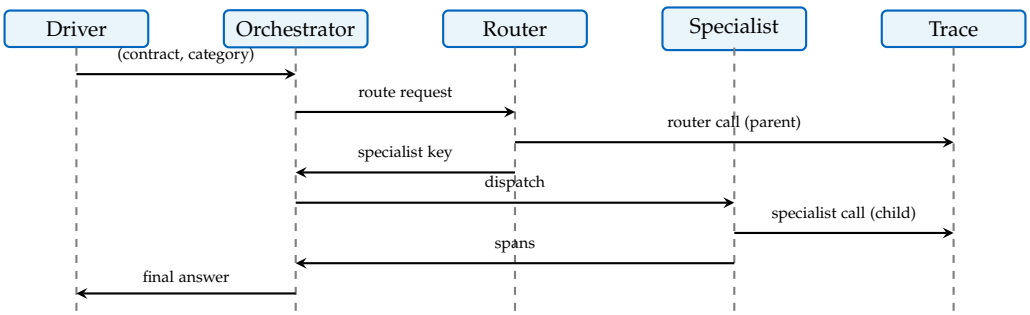


Figure 4.2: M1 sequence diagram for a single (contract, category) invocation. The router call is the Langfuse parent; the specialist call is its child.

M2: Consolidated specialist (architectural ablation). Its prompt is the textual union of the three M1 specialist prompts, concatenated into a single agent with no router, no orchestrator, and no per-category branching:

$$\texttt{prompt}_{\text{M2}} = \text{join}\Big(\texttt{prompt}_{\text{risk}},\ \texttt{prompt}_{\text{temporal}},\ \texttt{prompt}_{\text{ip}}\Big)$$

The three preambles are separated by domain headings; the verbatim-span output contract is stated once at the end. M2's purpose is methodological: M1 vs. M2 isolates routing and decomposition; M2 vs. B0 isolates prompt content. Together the two contrasts decompose the M1-vs-B0 delta into an architectural component and a

prompt-content component, which is the basis for the routing-redundancy finding in Section 5.4.

4.6 Observability and Trace Schema

What is recorded. For each clause-level decision the system captures: (i) configuration ID and model identifier; (ii) prompt template ID, version, and rendered prompt; (iii) full model completion; (iv) parsed final answer with parser version and status; (v) routing decision (M1) or classifier verdict (M0) where applicable; (vi) per-call token counts, billed cost, and wall-clock latency. M1 traces are parent–child structured: the router call is the parent, the specialist call is its child, and each call serialises the six fields above plus its `parent_id`, the model snapshot string, and a `parser_status` marker. A representative JSONL record is included verbatim in the released artefact bundle (§4.7).

Auditability properties. The trace structure supports three concrete properties. *Decision attribution*: any final answer is traceable to the specific agent, prompt version, and model that produced it. *Counterfactual replay*: because prompts and model identifiers are pinned per call at `temperature=0`, a reviewer can re-run a single archived call and obtain a deterministically identical completion. *Rejection forensics*: when M0 silently returns "no related clause", the classifier's verdict is recorded explicitly, making the classification-gate cascade of Section 5.5 visible at audit time as an upstream rejection rather than an unobservable absence.

Trade-offs and limits. The JSONL persistence layer is on the order of gigabytes per full sweep, and trace fields containing rendered prompts and full completions inherit the contract's confidentiality classification. The system distinguishes between decision-level fields (configuration, verdict, token counts) safe to surface to operators, and content-level fields (rendered prompts, full completions) that are access-controlled. The trace is faithful to the call graph, not to the model's internal computation: a chain-of-thought trace records what the model emitted, not what it computed.

Trace completeness. Trace completeness (the fraction of model calls for which all expected fields are populated) was 100% for B0, B1, and M2 (single-call configurations with no routing decisions), 100% for M0 (two-call pipeline with deterministic branching),

and 95.2% for M1 across official runs. The binding constraint is M1: the remaining 4.8% are provider-side truncations on long contracts and are flagged in the run logs. The single-configuration baseline (100%) confirms that the drop is attributable to M1's multi-call routing overhead, not to infrastructure failures.

4.7 Experimental Infrastructure

Loader and sampler. The CUAD loader is the single source of truth for which contracts and categories enter a run. It loads the 102-contract CUAD test split, applies the `neg_ratio=0.7` sampler of Section 3.2 (retains all positives, $\approx$60% of negatives), and assigns the prevalence-tier partition. It is deterministic under `seed=42` and produces the identical $\approx$2,999-sample envelope across all five configurations, making paired tests in Chapter 5 well-defined at the (contract, category) level.

Statistical aggregation. A separate aggregation pipeline reads per-run JSONL outputs, computes per-sample correctness vectors, and emits the McNemar contingency tables, paired bootstrap distributions, and BH-corrected p-value tables of Section 3.6. Aggregation is deterministic given the JSONL inputs and a fixed bootstrap seed; intermediate artefacts are persisted so results can be regenerated without rerunning models.

Dashboard. A Next.js dashboard supports interactive exploration: per-model F_2 comparisons, per-tier breakdowns, per-call cost and latency views, and trace inspection by sample ID. The over-extraction failure mode in Chapter 5 was first surfaced visually through the dashboard's per-category ω panel.

Test suite. An 81-test suite covers the metric implementations (F_1/F_2, Jaccard, λ, ω), the loader (filter, sampling determinism, tier assignment), the agent topologies (each of B0, B1, M0, M1, M2 has at least one end-to-end test against a stub model), and the statistical pipelines (McNemar, bootstrap, BH correction). The full suite passes prior to every official run.

Open-source release. The full source code, prompt registry, experiment configurations, and the experiment-results JSONL bundle are released as the open-source repository `agentic-ai-for-clm` at `https://github.com/ndtrung00/agentic-ai-for-clm`. The

repository contains every component referenced in this chapter: the LangGraph topologies for B0–M2, the YAML prompt files, the model client, the statistical pipeline, the dashboard, and the run logs underlying every table and figure in Chapter 5.

Reproducibility. Three commitments support reproducibility: deterministic sampling under `seed=42`; explicit version pinning of model identifiers (full provider snapshot strings) in the run metadata; and persistence of the full trace bundle (rendered prompts, completions, parsed answers, diagnostics) so that the statistical pipelines of Section 3.6 can be regenerated from raw logs without rerunning models. The prompt registry version hash is written to every run record, making prompt content auditable independently of the run logs.

5 Results

Across 55 (model, configuration) cells and 163,644 paired predictions, no multi-agent configuration improves pooled F_2 over the strongest single-agent baseline by more than 0.011 points; the dominant signal is the model-family $\times$ configuration interaction (Anthropic peaks under CoT, Google under Zero-Shot, OpenAI under Classify-Extract), with capability tier as a secondary moderator.

5.1 Run Coverage and Sample Accounting

The 55-cell grid yielded 164,941 raw JSONL records. After dropping 1,297 records (0.79%) with no usable classification token, 163,644 paired sample-level predictions remain. The dropped records concentrate in Zero-Shot (B0, n=838) and Classify-Extract (M0, n=362). The exclusion criterion is encoded in `src/evaluation/metrics.py`: a record is excluded if and only if its `evaluation.classification` field is not one of {`TP`, `FP`, `FN`, `TN`} after parsing.[1] Table 5.1 reports the analysed count per cell. All statistical tests use the paired intersection of sample IDs for the configurations being compared; intersection sizes are reported with each test. The positive rate across the retained set is 41.5%: the sampler retains all positive (contract, category) pairs and approximately 60% of negatives, upsampling positives relative to CUAD's natural $\approx$30% prior in order to preserve statistical power on rare-tier categories (§3.2).

[1] Inspection of the dropped records shows the failure mode is empty model completions (`raw_response = ""`) rather than structurally malformed JSON; the parser reaches no classification token because the model returned no token at all. The B0 concentration reflects Zero-Shot's stricter span acceptance criterion: B0 lacks the structured-output schema enforcement that M0/M1 specialist prompts provide, so an empty completion is unrecoverable for B0 but retried by the M0/M1 extractor downstream of the gate. The criterion is parser-version-stable because the four-value classification vocabulary is fixed in code, not configurable.

Table 5.1: Coverage: analysed predictions per (model, configuration) cell after dropping malformed records. *Partial run due to upstream sampling interruption; paired tests use the sample-ID intersection. Configuration names defined in Table 3.1. The two B0 partial cells are not missing-completely-at-random: GPT-4.1's 471 dropped records concentrate on five long contracts (backfilled median length ≈122,000 characters vs. ≈24,000 for retained records), consistent with provider-side API time-outs on the long-context tail; Sonnet 4's 359 dropped records spread across 98 of 102 contracts with a milder length skew (median ≈31,000 vs. ≈26,000). The contract-length moderation analysis (Section 5.8) shows multi-agent configurations underperform B0 on long contracts; for GPT-4.1 the missing-data direction therefore biases the Tab 5.4 ΔF_2 *toward* the multi-agent configurations and the per-model H1 verdict for GPT-4.1 understates rather than overstates the multi-agent disadvantage on the full evaluation envelope.

Model	**B0**	**B1**	**M0**	**M1**	**M2**	**Total**
Claude Haiku 4.5	2,999	2,999	2,999	2,999	2,999	14,995
Claude Opus 4.6	2,999	2,999	2,999	2,999	2,999	14,995
Claude Sonnet 4	2,640*	2,999	2,999	2,999	2,999	14,636
Claude Sonnet 4.6	2,999	2,999	2,999	2,999	2,999	14,995
Gemini 2.5 Flash	2,999	2,999	2,998	2,999	2,999	14,994
Gemini 2.5 Pro	2,999	2,999	2,999	2,999	2,999	14,995
Gemini 3 Flash	2,997	2,985	2,984	2,999	2,999	14,964
Gemini 3.1 Pro	2,994	2,976	2,978	2,999	2,998	14,945
GPT-4.1	2,528*	2,999	2,786	2,999	2,978	14,290
GPT-4.1 Mini	2,999	2,994	2,887	2,999	2,987	14,866
GPT-4.1 Nano	2,998	2,998	2,999	2,999	2,975	14,969
Total	32,151	32,946	32,627	32,989	32,931	**163,644**

5.2 H1: Multi-Agent vs. Single-Agent Baselines

Table 5.2 reports the headline metrics (F_2, F_1, recall, precision, and over-extraction rate ω) pooled across all eleven models, and per provider family.

The point estimates above are computed under the upsampled-positive class distribution (positive rate 41.5%, §3.2). Reweighting each negative-label sample by $w_{\text{neg}} = 0.7/0.585 \approx 1.196$ (equivalently, undoing the sampler's ≈ 60% negative-retention rate) recovers CUAD's natural ≈30% positive prevalence and yields, on the panel-pooled mean, $\overline{F_2}(\text{B0}) = 0.787$ and $\overline{F_2}(\text{M0}) = 0.788$ (delta +0.001 in M0's favour,

Table 5.2: Headline metrics by family and configuration (Cfg), mean across constituent models. ω = over-extraction rate (FP / negatives). Bold: best-in-column per family. Configuration names defined in Table 3.1. The F_2 and F_1 rankings diverge: pooled F_2 favours Zero-Shot (B0, 0.801) by 0.011 points; pooled F_1 favours Classify-Extract (M0, 0.798) by 0.061 points.

Family	**Cfg**	$\overline{F_2}$	$\overline{F_1}$	$\overline{R}$	$\overline{P}$	$\overline{\omega}$
Anthropic	B0	0.828	0.736	0.911	0.628	0.391
	B1	**0.851**	0.740	0.928	0.624	0.436
	M0	0.848	**0.826**	0.857	**0.808**	**0.162**
	M1	0.839	0.803	0.860	0.774	0.205
	M2	0.836	0.760	0.882	0.673	0.330
Google	B0	**0.826**	0.747	0.851	0.683	0.351
	B1	0.763	0.689	0.795	0.622	0.399
	M0	0.748	0.777	0.732	**0.844**	**0.099**
	M1	0.808	0.782	0.823	0.762	0.205
	M2	0.778	0.706	0.819	0.641	0.376
OpenAI	B0	0.731	0.725	0.737	0.722	**0.176**
	B1	0.768	0.701	0.815	0.628	0.365
	M0	**0.783**	**0.789**	0.781	**0.798**	0.138
	M1	0.700	0.735	0.696	0.787	0.111
	M2	0.744	0.698	0.762	0.645	0.306
All (11)	B0	**0.801**	0.737	**0.854**	0.666	0.320
	B1	0.796	0.711	0.871	0.611	0.385
	M0	0.794	**0.798**	0.793	**0.813**	**0.130**
	M1	0.790	0.776	0.801	0.768	0.179
	M2	0.790	0.724	0.844	0.642	0.339

vs. -0.007 under upsampling); $\overline{F_1}(\text{M0})$ widens its lead from $+0.061$ to $+0.073$. Both reweighted deltas remain inside the $|\Delta F_2| \geq 0.025$ inconclusive band of §3.6, and the qualitative H1 verdict is therefore preserved across the two distributions; what shifts is the direction of the small pooled F_2 gap.[2]

The pooled means tell two stories. On F_2, Zero-Shot (B0) is nominally best at 0.801 with all five configurations within a 0.011-point band. On F_1, Classify-Extract (M0) is best at 0.798 with a 0.087-point spread. The divergence is mechanistic: B0 has the highest recall (0.854) and the second-highest over-extraction rate (0.320), while M0 has the highest

[2]Per-model reweighted point estimates are released alongside the experiment artefacts (§4.7); the reweighting is applied at the metric level and does not require re-running model inference.

precision (0.813) and the lowest over-extraction rate (0.130). The β=2 weighting of F_2 rewards Zero-Shot's recall; F_1's equal weighting rewards Classify-Extract's calibration.

Figure 5.1 visualises the model-family interaction: each family peaks under a different configuration.

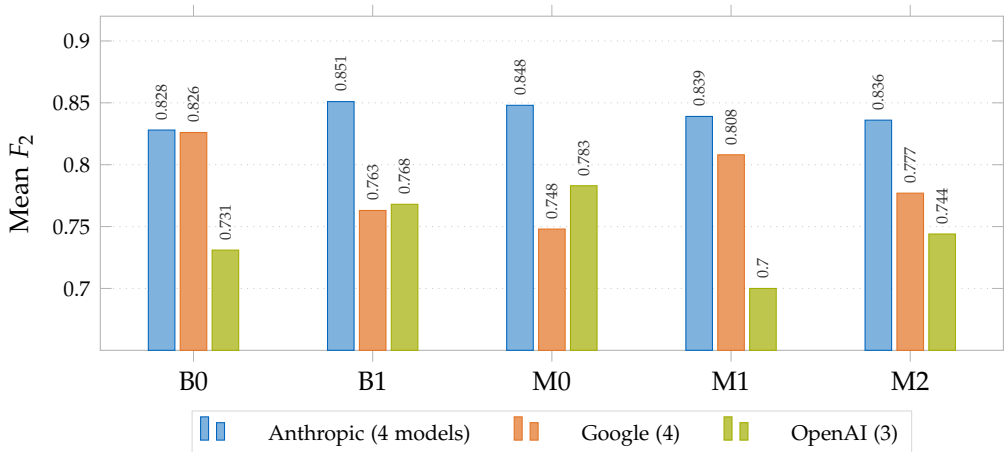


Figure 5.1: Mean F_2 by provider family × configuration. Each family peaks under a different configuration: Anthropic under CoT (B1, 0.851), Google under Zero-Shot (B0, 0.826), OpenAI under Classify-Extract (M0, 0.783). No single architecture is universally optimal. The family-by-configuration interaction is the dominant signal in the H1 results.

Table 5.3 reports per-model F_2. No configuration is uniformly best: B0 wins on 4 models, M0 on 4, B1 on 2, and M1 on 1. The ranking also varies with model compute tier (Figure 5.2): Classify-Extract leads on light- and mid-tier models, while Zero-Shot leads on heavy models, qualifying contribution C2.

Table 5.4 reports the H1 paired tests. Each row is one multi-agent vs. B0 contrast on the sample-ID intersection. McNemar exact p-values are computed via Eq. 3.1 and corrected for the m=22-test family by Benjamini–Hochberg (Eq. 3.2); paired-bootstrap CIs use B=300, `seed=42` (Algorithm 1). CIs are computed on the paired sample-ID intersection; for models with partial runs (Claude Sonnet 4, GPT-4.1) the intersection size is smaller than the full run, so the intersection-based ΔF_2 may differ slightly from the full-run values in Table 5.3.

H1 verdict. Under the pre-registered decision rule (§3.5), Classify-Extract (M0) satisfies the per-config BH-significant-positive majority condition (6 of 11 contrasts: Haiku,

Table 5.3: F_2 by model and configuration. Bold: best per row. Configuration names defined in Table 3.1. The family interaction is pronounced: the Anthropic family peaks under CoT (B1), the Google family under Zero-Shot (B0), and the OpenAI family under Classify-Extract (M0).

Model	F_2(B0)	F_2(B1)	F_2(M0)	F_2(M1)	F_2(M2)
Claude Haiku 4.5	0.783	**0.849**	0.792	0.835	0.822
Claude Opus 4.6	0.863	**0.865**	0.864	0.860	0.846
Claude Sonnet 4	0.851	0.851	**0.877**	0.826	0.842
Claude Sonnet 4.6	0.813	0.840	**0.860**	0.834	0.836
Gemini 2.5 Flash	**0.829**	0.818	0.619	0.810	0.832
Gemini 2.5 Pro	0.828	0.810	0.707	**0.834**	0.816
Gemini 3 Flash	0.786	0.580	**0.822**	0.747	0.609
Gemini 3.1 Pro	**0.861**	0.843	0.843	0.840	0.854
GPT-4.1	**0.868**	0.852	0.828	0.777	0.853
GPT-4.1 Mini	0.837	0.841	**0.862**	0.777	0.833
GPT-4.1 Nano	0.488	0.612	**0.659**	0.547	0.545
Mean	**0.801**	0.796	0.794	0.790	0.790

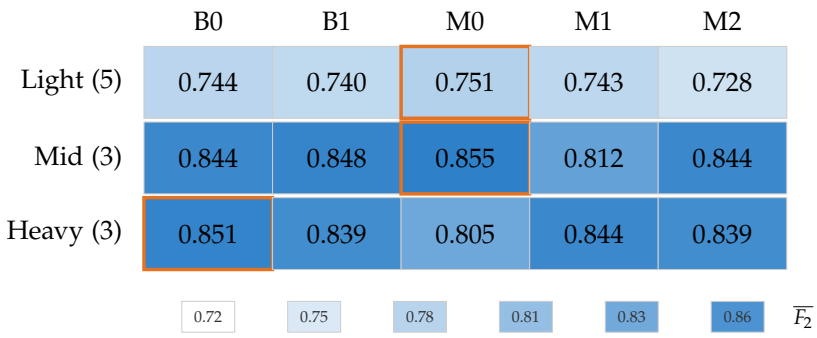


Figure 5.2: Capability tier × configuration heatmap of mean F_2. Cells coloured by $\overline{F_2}$; orange outline marks the best configuration per tier. Classify-Extract (M0) wins on Light and Mid tiers; Zero-Shot (B0) wins on Heavy. Decomposition narrows the Heavy–Light capability gap from 0.106 under B0 to 0.054 under M0 (49% reduction); the same decomposition penalises Heavy models on F_2 because they already absorb long-context load in a single call (see §6.1).

Table 5.4: H1 paired tests: $\Delta F_2 = F_2(\text{multi}) - F_2(\text{B0})$ computed on the sample-ID intersection (so the row ΔF_2 may differ from the full-run difference implied by Table 5.3 when one of the pair is a partial run, namely Claude Sonnet 4 and GPT-4.1). Paired-bootstrap 95% CIs (B=300, `seed=42`), BH-corrected significance (α=0.05, m=22). ✓: BH-reject H_0. Note: BH significance is decided by the McNemar exact p-value; the bootstrap CI on ΔF_2 is reported separately and may not align with the McNemar verdict on near-zero effects where the discordant-pair distribution and the resampled-mean distribution diverge.

	M0 vs. B0			**M1 vs. B0**		
Model	ΔF_2	CI_{95}	BH	ΔF_2	CI_{95}	BH
Claude Haiku 4.5	+0.009	[−0.009, +0.026]	✓	+0.052	[+0.038, +0.066]	✓
Claude Opus 4.6	+0.001	[−0.011, +0.014]	✓	−0.004	[−0.017, +0.009]	✓
Claude Sonnet 4	+0.025	[+0.016, +0.043]	✓	−0.025	[−0.031, −0.001]	✓
Claude Sonnet 4.6	+0.047	[+0.032, +0.062]	✓	+0.021	[+0.002, +0.038]	✓
Gemini 2.5 Flash	−0.210	[−0.234, −0.181]	✓	−0.020	[−0.036, −0.003]	✓
Gemini 2.5 Pro	−0.121	[−0.143, −0.094]	✓	+0.006	[−0.006, +0.020]	✓
Gemini 3 Flash	+0.037	[+0.015, +0.055]	✓	−0.039	[−0.060, −0.014]	✓
Gemini 3.1 Pro	−0.018	[−0.035, −0.006]	✓	−0.021	[−0.034, −0.007]	×
GPT-4.1	−0.040	[−0.061, −0.026]	✓	−0.060	[−0.083, −0.042]	✓
GPT-4.1 Mini	+0.025	[+0.011, +0.035]	×	−0.060	[−0.081, −0.039]	×
GPT-4.1 Nano	+0.171	[+0.143, +0.195]	✓	+0.059	[+0.025, +0.091]	✓

Opus, Sonnet 4, Sonnet 4.6, Gemini 3 Flash, GPT-4.1 Nano) but Multi-Agent (Full) (M1) does not (4 of 11: Haiku, Sonnet 4.6, Gemini 2.5 Pro, GPT-4.1 Nano); H1 is rejected because the rule requires both configurations to clear the majority threshold. Across all 22 contrasts, Cohen's g is uniformly positive (range +0.005 to +0.363, median $\approx$ 0.20): multi-agent configurations win on raw paired-binary correctness on every single contrast, and the H1 rejection is driven by F_2's recall-weighted denominator inverting precision gains because correctly identifying a true negative does not enter the F_2 numerator. Conditional improvements survive: the OpenAI family benefits from M0 (GPT-4.1 Nano +0.171; pooled +0.052), and lighter Anthropic models benefit from M0/B1 (Sonnet 4.6 M0 +0.047). The model-family interaction is the dominant signal, with pooled F_2 within a 0.011-point band. On F_1, M0 leads pooled (0.798 vs. 0.737), reversing the F_2 ranking and making the architectural choice metric-dependent (the empirical β flip-point is examined in §6.4; §5.9 gives the one-line RQ answer).

5.3 H2: Rare-Category Gap

Table 5.5 reports the rare–common gap $g = F_2^{\text{common}} - F_2^{\text{rare}}$ for each (model, configuration) cell. Mean common-tier F_2 stays in a narrow band across configurations (Zero-Shot 0.929, CoT 0.912, Classify-Extract 0.878, Multi-Agent (Full) 0.940, Multi-Agent (Prompt-Only) 0.919); the gap movement is driven almost entirely by rare-tier shifts.

Table 5.5: Rare–common gap $g = F_2^c - F_2^r$ per (model, configuration). Lower is better. Classify-Extract's mean gap (0.284) exceeds Zero-Shot's (0.276) due to gate-induced rare-tier collapse on Gemini 2.5 Flash/Pro. Multi-Agent (Full)'s mean gap (0.342) is worst, driven by GPT-family specialist failure on rare categories. Only CoT and Multi-Agent (Prompt-Only) reduce the gap relative to Zero-Shot (means 0.240 and 0.226).

Model	g(B0)	g(B1)	g(M0)	g(M1)	g(M2)
Claude Haiku 4.5	0.329	0.263	**0.074**	0.296	0.306
Claude Opus 4.6	0.219	**0.169**	0.202	0.260	0.256
Claude Sonnet 4	0.245	0.241	0.211	0.257	**0.195**
Claude Sonnet 4.6	0.269	**0.149**	0.220	0.245	0.175
Gemini 2.5 Flash	0.216	0.227	0.344	0.277	**0.153**
Gemini 2.5 Pro	**0.268**	0.272	0.515	0.258	0.323
Gemini 3 Flash	0.324	0.337	0.355	0.346	**0.144**
Gemini 3.1 Pro	0.225	0.236	0.289	0.262	**0.189**
GPT-4.1	0.246	0.220	**0.179**	0.533	0.219
GPT-4.1 Mini	**0.196**	0.254	0.233	0.421	0.293
GPT-4.1 Nano	0.296	0.280	0.296	0.608	**0.234**
Mean	0.276	0.240	0.284	0.342	**0.226**

H2 verdict. H2 is rejected. Under the pre-registered decision rule (§3.5; upper bound of the 95% paired-bootstrap CI on ΔGap strictly < 0), only 1 of the 22 multi-agent vs. B0 contrasts satisfies the rule (Claude Haiku 4.5 under M0, ΔGap$= -0.256$, CI $[-0.308, -0.211]$); 6 contrasts have CIs strictly above zero: M0 and M1 *widen* the gap significantly on the Gemini and GPT-family models. Because the decision rule is a one-sided CI inclusion criterion rather than a p-value, the Benjamini–Hochberg procedure (Eq. 3.2) does not apply mechanically across the 22 contrasts; the family-wise Type-I rate is therefore not formally controlled, and the 1-of-22 single-rule satisfaction would survive any reasonable family-wise correction at α=0.05 (the verdict cannot

become more permissive under correction). Per-model bootstrap CIs are released in the experiment artefacts. Two mechanisms drive the widening. Under M0, the classification gate (Section 5.5) collapses rare-tier F_2 on the Gemini family (Gemini 2.5 Flash: F_2^r drops from 0.719 under B0 to 0.381 under M0). Under M1, GPT-family specialists fail catastrophically on rare categories (GPT-4.1: F_2^r drops 0.704 $\rightarrow$ 0.430; GPT-4.1 Nano: 0.253 $\rightarrow$ 0.167). The B1 chain-of-thought baseline and M2 consolidated specialist do reduce the gap (means 0.240 and 0.226), but neither is a multi-agent architecture in the sense of Chapter 3.

Figure 5.3 disaggregates by prevalence tier to expose which configuration wins at each tier rather than on a gap-adjusted average. The pattern reverses across tiers: M1 leads on common categories, M0 on moderate, and B0 on rare. Multi-agent specialisation therefore benefits common-tier categories more than rare-tier ones (Figure 5.3).[3]

[3]The rare-tier mean masks per-category heterogeneity. M0 leads decisively on specific rare categories where the gate's precision gain compensates the recall cost: *Source Code Escrow* (F_2=1.000 vs. B0 0.943), *Joint IP Ownership* (+0.142 over B0), *No-Solicit of Customers* (+0.233). The tier-mean ranking is dominated by Gemini-family cascade collapse on a subset of rare categories (§5.5), not by uniform M0 weakness across the rare tier.

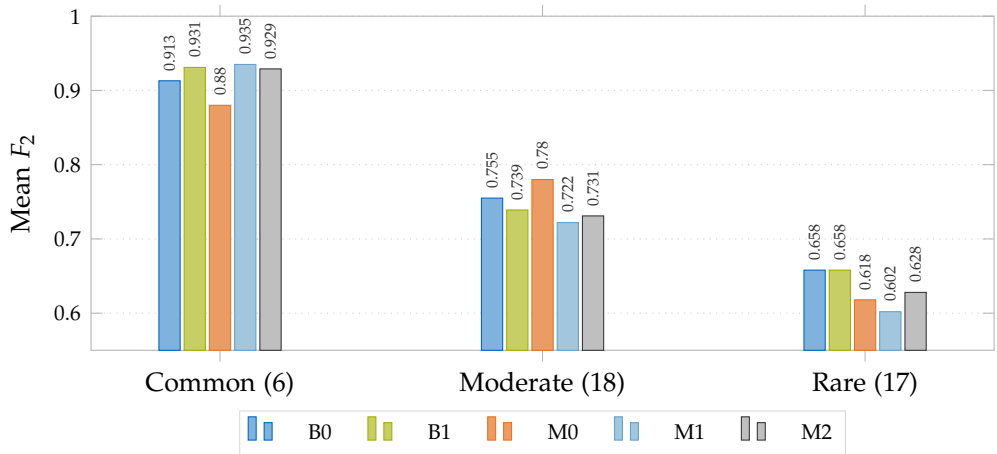


Figure 5.3: Mean F_2 by CUAD prevalence tier $\times$ configuration, pooled across the eleven-model panel. Tier partition derived from positive-instance prevalence in the 102-contract test split (common $>$ 30% positive, moderate 10–30%, rare $<$ 10%); marginally different from the ContractEval F1-based partition (6/14/21) without changing the qualitative ranking. The best configuration per tier differs: M1 wins common (0.935), M0 wins moderate (0.780), B0 and B1 tie on rare (0.658). Multi-agent decomposition helps on categories where the base model is already strong; H2 is rejected because it predicts the opposite direction for rare categories.

5.4 H3: Architecture vs. Prompt Content (Multi-Agent (Full) vs. Multi-Agent (Prompt-Only))

The Multi-Agent (Full) (M1) router agreed with the deterministic category-to-specialist mapping on 100.00% of requests for ten of eleven models, and on 99.03% for GPT-4.1 Nano (29 mis-routes out of 2,999), yielding 99.91% routing accuracy across the panel. The routed-call distribution across specialists is uneven: `ip_commercial` 11,802 calls, `temporal_renewal` 11,364, `risk_liability` 9,068. The pattern reflects the per-specialist category populations (17, 11, 13) and CUAD's prevalence-weighted sampling rather than a routing defect. Routing is empirically a deterministic lookup: the router's decisions can be replicated by a static dictionary mapping category names to specialist labels at zero LLM cost. We therefore present no figure for routing accuracy and treat M1 vs. M2 as a test of whether dispatch architecture (with routing held empirically constant) adds value over a single-agent prompt containing the same specialist text.

Table 5.6 reports the M1–M2 contrasts on F_2 and on average Jaccard span overlap.

Table 5.6: H3 contrasts: Multi-Agent (Full) (M1) vs. Multi-Agent (Prompt-Only) (M2). Both configurations are tied on pooled F_2 ($\overline{\Delta F_2} = -0.0002$; 5 M1 wins, 6 M2 wins). On span quality (Jaccard), Multi-Agent (Prompt-Only) dominates on 10 of 11 models, mean advantage +0.071. BH: BH-corrected significance (α=0.05, m=11). All 11 contrasts are significant. The single M1 exception on Jaccard is Gemini 3 Flash, where Multi-Agent (Prompt-Only) collapses (Section 5.5).

Model	F_2(M1)	F_2(M2)	ΔF_2	BH	Jacc. (M1)	Jacc. (M2)
Claude Haiku 4.5	0.835	0.822	+0.013	✓	0.403	**0.449**
Claude Opus 4.6	0.860	0.846	+0.014	✓	0.429	**0.492**
Claude Sonnet 4	0.826	0.842	−0.016	✓	0.491	**0.524**
Claude Sonnet 4.6	0.834	0.836	−0.002	✓	0.397	**0.460**
Gemini 2.5 Flash	0.810	0.832	−0.022	✓	0.384	**0.462**
Gemini 2.5 Pro	0.834	0.816	+0.018	✓	0.375	**0.440**
Gemini 3 Flash	0.747	0.609	+0.138	✓	**0.345**	0.332
Gemini 3.1 Pro	0.840	0.854	−0.014	✓	0.420	**0.534**
GPT-4.1	0.777	0.853	−0.076	✓	0.298	**0.445**
GPT-4.1 Mini	0.777	0.833	−0.056	✓	0.319	**0.387**
GPT-4.1 Nano	0.547	0.545	+0.001	✓	0.272	**0.396**
Mean	0.790	0.790	−0.000	—	0.376	**0.447**

Despite identical routing, Multi-Agent (Full) (M1) and Multi-Agent (Prompt-Only) (M2) are statistically distinguishable on F_2 for all 11 models after BH correction (McNemar two-sided, α=0.05, m=11; Table 5.6). The direction is inconsistent: M1 outperforms M2 on 5 models, M2 on 6. There is no systematic family pattern. On the secondary axis of span quality, Multi-Agent (Prompt-Only) dominates 10 of 11 models with a +0.071 Jaccard advantage, indicating that the consolidated single-agent prompt produces more precise, tighter extractions than the routed specialist pipeline (contribution C4).

H3 verdict. H3 is supported at the per-model level: Multi-Agent (Full) and Multi-Agent (Prompt-Only) are statistically distinguishable on F_2 for all 11 models after BH correction (McNemar two-sided, α=0.05, m=11). The direction is inconsistent: M1 outperforms M2 on 5 models, M2 on 6, and pooled F_2 is identical to three decimal places (0.790 vs. 0.790). H3 therefore confirms that dispatch architecture alters model behaviour, not that it improves it. The direction inconsistency is itself a finding: routing overhead hurts precision-oriented models (GPT-family, $\Delta F_2 = -0.056$ to -0.076) and

helps recall-deficient ones (Gemini 3 Flash, $\Delta F_2 = +0.138$). On the secondary axis of span quality, M2 dominates 10 of 11 models with a +0.071 Jaccard advantage (contribution C4): the single-agent consolidated prompt produces tighter, more precise extractions than the routed specialist pipeline. A deterministic lookup replaces the LLM router without measurable F_2 loss.

5.5 Classification-Gate Cascade

Table 5.7 quantifies the M0 cascade. The M0 classifier's `predicted_no_clause` decision bypasses the extractor and records "no related clause" as the final answer. We report the FN-rate (FN/(TP + FN)) for B0 and M0, and the share of M0 false negatives attributable to a gate rejection rather than a downstream extraction failure.

Table 5.7: M0 classification-gate rejection cascade. FN-rate = FN/(TP + FN). Gate-FN: false negatives where the classifier emitted `predicted_no_clause`. Gate % = Gate-FN / M0 FN. On average 82.7% of M0's false negatives originate at the gate; Gemini 2.5 Flash and Pro lose ≥97.8% of their recall to upstream rejection.

Model	**B0 FN-rate**	**M0 FN-rate**	**M0 FN**	**Gate-FN**	**Gate %**
Claude Haiku 4.5	13.4%	20.6%	255	210	82.4%
Claude Opus 4.6	6.9%	11.5%	142	95	66.9%
Claude Sonnet 4	7.9%	9.9%	123	87	70.7%
Claude Sonnet 4.6	10.9%	12.6%	156	131	84.0%
Gemini 2.5 Flash	9.6%	42.2%	520	509	97.9%
Gemini 2.5 Pro	7.9%	32.6%	403	394	97.8%
Gemini 3 Flash	17.2%	17.9%	221	197	89.1%
Gemini 3.1 Pro	9.9%	15.5%	191	169	88.5%
GPT-4.1	9.2%	16.6%	207	191	92.3%
GPT-4.1 Mini	13.4%	11.8%	147	103	70.1%
GPT-4.1 Nano	55.5%	36.3%	452	318	70.4%
Mean	14.7%	20.7%	256	218	82.7%

The cascade is the dominant cause of M0's F_2 regression on the Google family. On Gemini 2.5 Flash, M0 FN-rate jumps to 42.2% from B0's 9.6% (a 4.4× amplification), and 97.9% of those FNs are gate-attributable. The cascade interacts with category prevalence: rare-tier F_2 on Gemini 2.5 Flash drops from 0.719 (B0) to 0.381 (M0), a 0.338-point loss that fully accounts for the rare-gap widening in Section 5.3.

The cascade is a recall–precision exchange, not pure regression. The complement of lost recall is gained precision. Table 5.8 reports ω, the over-extraction rate (Eq. 2.5). Across the panel, M0 cuts mean ω from 0.320 (B0) to 0.130 (a 59% reduction in false positives on negatively-labelled instances). The Gemini family benefits most (ω drops from 0.352 to 0.099) but pays the steepest recall cost. M0 therefore trades recall for precision via the gate. The trade is favourable on F_1 (M0 leads pooled, Table 5.2) and unfavourable on F_2 (B0 leads pooled). Which side of the trade is preferable depends on the deployment cost ratio of missed clauses vs. false extractions, a question revisited in Chapter 6.

Table 5.8: Over-extraction rate ω (Eq. 2.5): fraction of negative-label instances on which the model emits a non-empty span. Bold: best per row. The classification gate cuts ω by more than half on average; the Gemini family gains the most precision at the recall cost documented in Table 5.7. CoT (B1) raises ω above Zero-Shot (B0) on every model, evidence that chain-of-thought prompting amplifies over-extraction; Multi-Agent (Full) (M1) stays close to Classify-Extract (M0) because it inherits the same specialist scaffold; Multi-Agent (Prompt-Only) (M2) reverts to the free-text baseline.

Model	ω(B0)	ω(B1)	ω(M0)	ω(M1)	ω(M2)
Claude Haiku 4.5	.478	.369	**.158**	.191	.316
Claude Opus 4.6	.330	.436	**.169**	.184	.335
Claude Sonnet 4	.335	.400	**.169**	.248	.236
Claude Sonnet 4.6	.423	.538	**.150**	.198	.433
Gemini 2.5 Flash	.392	.422	**.082**	.182	.354
Gemini 2.5 Pro	.456	.604	**.076**	.303	.454
Gemini 3 Flash	.321	.330	**.120**	.135	.450
Gemini 3.1 Pro	.236	.242	**.116**	.199	.242
GPT-4.1	.221	.459	**.102**	.119	.335
GPT-4.1 Mini	.223	.455	.174	**.155**	.409
GPT-4.1 Nano	**.085**	.181	.137	.057	.175
Mean	.320	.385	**.130**	.179	.339

Cascade predictability. The cascade severity correlates with the base model's B0 over-extraction rate: the more liberally a model extracts under B0, the more recall it surrenders at the gate. Gemini 2.5 Pro ($\omega_{B0} = 0.456$) loses 0.121 F_2 points to M0; GPT-4.1 Nano ($\omega_{B0} = 0.085$) *gains* 0.171 F_2 points, because the gate corrects its B0 laziness (55.5% FN-rate). A practitioner can predict whether M0 will help or hurt a

given model by running a cheap B0 calibration pass and inspecting ω_{B0} against the FN-rate profile. Figure 5.4 visualises the cascade as a movement in the (ω, λ) plane: arrows connect each model's B0 cell to its M0 cell, and the up-and-leftward shift on Gemini Flash/Pro is the cascade.

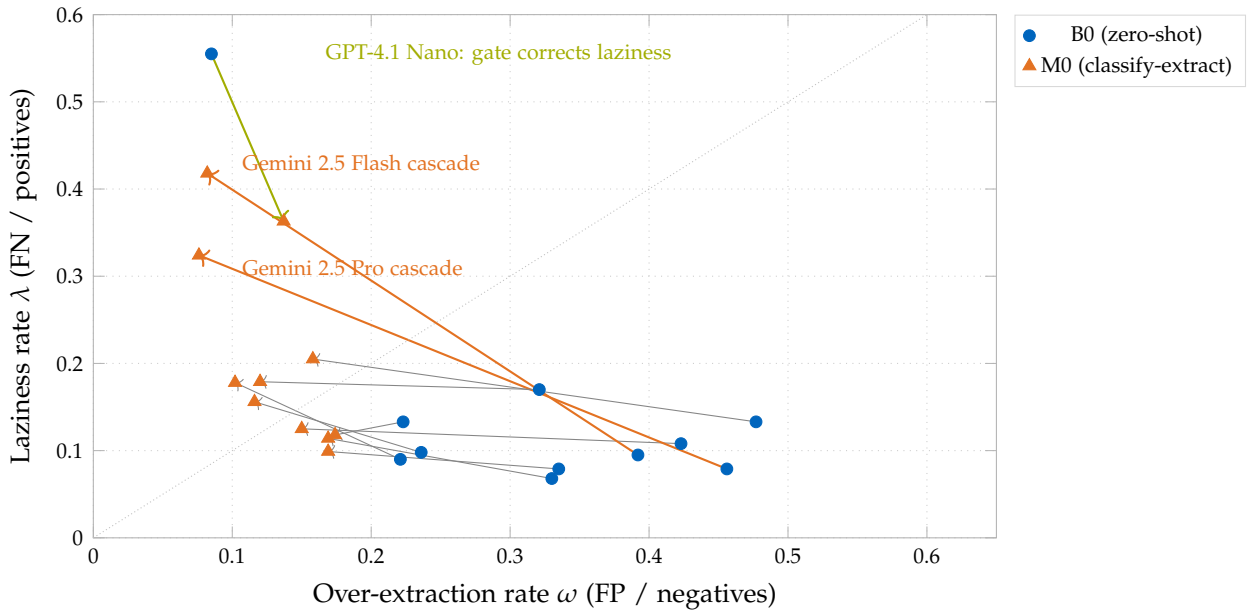


Figure 5.4: Recall–precision exchange under Classify-Extract. Each B0 model (•) maps to its M0 counterpart (▲); arrows show the per-model trajectory in the (ω, λ) plane. Most models move *left and slightly up*: a moderate precision gain (lower ω) paid with a small recall loss (higher λ). On Gemini 2.5 Flash and Pro (orange arrows) the recall loss is severe: λ jumps to > 0.32, the cascade documented in Table 5.7. On GPT-4.1 Nano (green arrow) the trajectory reverses: the gate corrects the model's high B0 laziness (λ=0.555 B0 $\rightarrow$ 0.363 M0). The dotted diagonal marks $\omega = \lambda$ as an orientation reference.

5.6 Span Grounding Rate

The grounding rate measures the fraction of an extraction's character span that is reproducible verbatim from the source contract; an ungrounded span is either paraphrased or hallucinated. Two views are reported in Table 5.9: pooled across all sampled instances (denominator includes the $\approx$58% negative-label cases where the model returns "no

related clause" and grounding is undefined, conventionally scored 0), and restricted to instances on which the model emitted at least one span.

Table 5.9: Mean grounding rate by configuration. *All*: pooled over all ≈2,999 samples per cell, including "no related clause" returns scored 0. *Predicted-clause only*: conditioned on a non-empty prediction. Multi-Agent (Full) (M1) achieves 1.000 pooled across every model in the predicted-clause column: the structured specialist output enforces contract-text citation. Classify-Extract (M0) reaches 0.893. Zero-Shot (B0) and Multi-Agent (Prompt-Only) (M2), which return free-text extractions, leave 44–58% of predicted spans ungrounded.

Config	**B0**	**B1**	**M0**	**M1**	**M2**
All samples	0.299	0.373	0.375	**0.456**	0.248
Predicted-clause only	0.543	0.594	0.893	**1.000**	0.415

The predicted-clause column is the audit-relevant denominator: a missed clause has no span to ground, but every emitted span is auditable in principle. Under that view, Multi-Agent (Full)'s structured-output schema (§4.6) guarantees verbatim citation by construction, and Classify-Extract's extractor inherits the same scaffold from the M1 specialist prompts. The two single-call generative configurations Zero-Shot and Multi-Agent (Prompt-Only) leave roughly half of their extractions paraphrased or hallucinated. The dominant failure mode is extraction of *plausible* legal text that cannot be located verbatim in the source contract. This finding qualifies contribution C4: M1's auditability surplus comes from two mechanisms, the trace structure (Section 4.6) and schema-enforced grounding.

5.7 Cost-Efficiency Frontier

Table 5.10 reports per-run provider costs and mean ratios relative to B0.

Figure 5.5 plots all 55 cells in the (F_2, cost) plane. B0 occupies the Pareto frontier at every cost level at which it appears. The three Gemini M0 cells are strictly Pareto-dominated: lower F_2 *and* higher cost than B0 Gemini alternatives.

The 23–67% mean cost premium attached to multi-agent configurations (measured under per-call provider billing without prompt caching) is not compensated by aggregate F_2 gains. In workflows where per-agent traces add operational value (obligation tracking, audit-trail generation), this premium may be acceptable; Chapter 6 elaborates

Table 5.10: Provider cost per run (≈2,999 samples) and per-model mean ratio relative to B0. M0's 1.67× premium reflects the additional classifier call; M1's 1.23× reflects the orchestrator hop.

Model	B0 ($)	B1 ($)	M0 ($)	M1 ($)	M2 ($)
Claude Haiku 4.5	36.70	43.11	60.66	45.00	43.29
Claude Opus 4.6	185.02	213.30	302.96	215.49	211.10
Claude Sonnet 4	94.04	125.98	184.58	127.17	125.41
Claude Sonnet 4.6	111.44	129.59	184.11	137.75	128.24
Gemini 2.5 Flash	10.64	14.72	13.73	13.31	13.99
Gemini 2.5 Pro	43.39	61.77	58.69	55.57	58.21
Gemini 3 Flash	16.55	19.09	26.24	19.65	19.18
Gemini 3.1 Pro	65.75	72.46	104.57	78.73	72.69
GPT-4.1	38.07	77.88	91.50	56.11	73.68
GPT-4.1 Mini	13.17	15.78	21.42	13.22	15.78
GPT-4.1 Nano	3.17	3.64	5.10	3.64	3.63
Mean ratio	1.00×	1.30×	1.67×	1.23×	1.27×

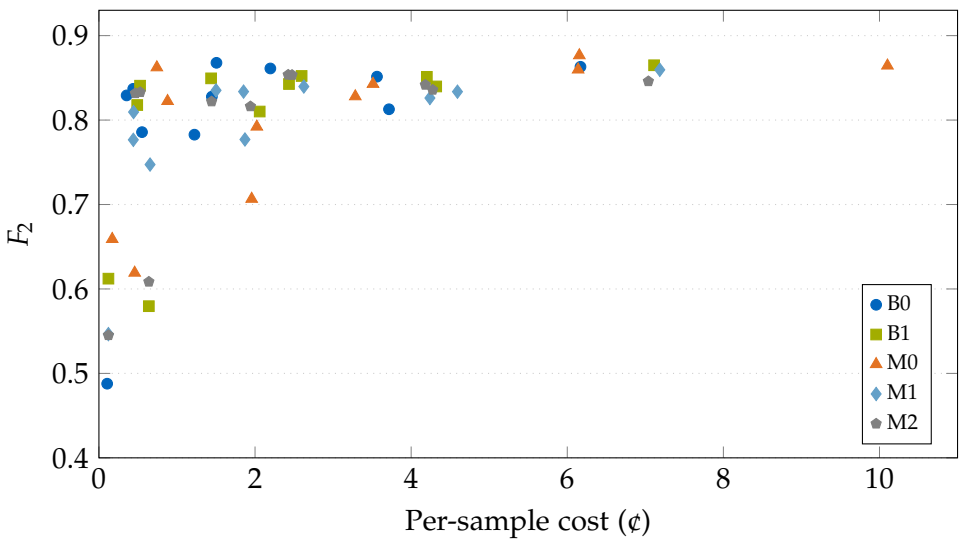


Figure 5.5: F_2 vs. per-sample cost across all 55 cells. The Pareto frontier is occupied by B0 at every cost level. The three Gemini M0 cells (Gemini 2.5 Flash, Gemini 2.5 Pro, and Gemini 3 Flash) are strictly Pareto-dominated: lower F_2 *and* higher cost than B0 alternatives.

these conditions. The premium estimate is sensitive to the cost model: prompt caching, which every provider in the panel supports, would shrink it toward the orchestrator-hop overhead alone (§7.4).

5.8 Contract-Length Moderation

The cascade analysis (Section 5.5) and tier analysis (Figure 5.3) treat all contracts uniformly. Contract length provides an orthogonal moderating variable: the context-window motivation for multi-agent decomposition predicts that routing clauses to specialists should help on *longer* contracts, where a monolithic prompt must attend over more irrelevant text. We test this prediction by splitting all 102 CUAD test contracts into quartiles by character count and computing per-configuration F_2 within each quartile. The main evaluation already uses all 102 contracts of the CUAD test split, so the quartile analysis here is a length-stratified view of the headline results in Sections 5.2–5.5 rather than a separate evaluation envelope.

Table 5.11: Mean F_2 by contract-length quartile (102 contracts, ≈26 per quartile), pooled across all eleven models. Quartile boundaries: 14,384 / 25,922 / 62,272 characters. Multi-agent configurations (M0, M1) outperform B0 on short contracts (Q1–Q2) by +1.7 to +3.7 percentage points and underperform on long contracts (Q3–Q4) by −0.5 to −4.1 percentage points. The pattern is opposite to the context-window motivation.

Quartile	Range (chars)	B0	B1	M0	M1	Best
Q1 (shortest)	0–14,384	0.756	0.767	**0.776**	**0.793**	M1
Q2	14,385–25,922	0.791	0.803	**0.808**	0.800	M0
Q3	25,923–62,272	0.806	0.793	0.801	0.799	B0
Q4 (longest)	>62,272	**0.849**	0.819	0.817	0.808	B0

Table 5.11 shows the opposite of the predicted pattern. On Q1 contracts (under 14,384 characters), M1 leads by +3.7 percentage points over B0 and M0 by +2.0 points. On Q4 contracts (over 62,272 characters), B0 leads by +3.2 points over M0 and +4.1 over M1. The crossing point lies between Q2 and Q3 (≈25,900 characters). The mechanism is plausibly gate-induced: on long contracts the M0 classifier is exposed to larger token windows, increasing the probability of conservative `predicted_no_clause` decisions that trigger the cascade documented in Section 5.5. For M1, long-contract degradation may reflect specialist prompt dilution when many clause types from one domain

appear in a single long document. Multi-agent decomposition as implemented offers no context-window advantage over a zero-shot prompt on CUAD-class contract lengths.

5.9 Answers to Research Questions

The full RQ answers are stated in Chapter 7, §7.2; this section gives the one-line summary that the results justify.

RQ1. No architecture wins universally on F_2 (pooled spread 0.011); the model-family interaction is dominant (Anthropic peaks at B1, Google at B0, OpenAI at M0), and the metric choice flips the ranking (F_1: M0 leads pooled; F_2: B0 leads pooled).

RQ2. Under per-call provider billing without prompt caching, B0 occupies the Pareto frontier at every cost level (Figure 5.5); multi-agent cost premiums of 1.23–1.67× do not pay back in pooled F_2. Under prompt-caching cost models, the premium shrinks toward the orchestrator-hop overhead and the frontier may invert (§7.4).

RQ3. M0 and M1 produce the trace primitives B0 cannot (§4.6, Table 5.9); the 4× token volume, the routing redundancy (99.91% deterministic), and the bounded attribution faithfulness are the costs.

6 Discussion

No multi-agent configuration improves on the strongest single-agent baseline by more than 0.011 pooled F_2 points across eleven models.

6.1 The Headline Finding and Its Conditions

On the 102-contract CUAD test split under the upsampled-positive class distribution described in §3.2, no multi-agent configuration improves on the strongest single-agent baseline by more than 0.011 points of mean F_2 pooled across the eleven-model panel. Under the pre-registered decision rule of Section 3.5, H1 is rejected in its universal form: no multi-agent configuration achieves consistent superiority across the eleven-model panel, and eleven of the 22 per-model contrasts are negative or zero. Conditional improvement exists for individual (model, configuration) cells (Classify-Extract (M0) on the OpenAI family and lighter Anthropic models), but the pooled signal is dominated by the model-family interaction. H2 is rejected: both pipeline decomposition (Classify-Extract, M0) and specialist routing (Multi-Agent (Full), M1) widen the rare–common F_2 gap relative to Zero-Shot (B0). H3 is supported per model: Multi-Agent (Full) (M1) and Multi-Agent (Prompt-Only) (M2) are statistically distinguishable on every model after BH correction. The direction, however, is inconsistent across the panel, and the pooled F_2 difference is zero to three decimals.

The single most informative pattern is the model-family interaction. The Anthropic family achieves its peak under CoT (B1, $\overline{F_2}$=0.851); the Google family under Zero-Shot (B0, 0.826); the OpenAI family under Classify-Extract (M0, 0.783). The capability-tier gradient in Figure 5.2 is consistent with this: light- and mid-tier models gain from M0's calibration, while heavy-tier models are better served by B0, with the F_2 capability gap between Heavy and Light tiers narrowing from 0.106 under B0 to 0.054 under M0 (a 49% reduction). Decomposition acts partly as a capability-equalising effect, but at a cost (Section 6.2) that is not recovered for heavy models that already absorb long-context load in a single call. Prior work has observed that a well-prompted single agent matches multi-agent decomposition under stronger base models [Gao+25]; the

present data reproduce this: the Anthropic Opus tier achieves its best F_2 under B1, not under any decomposed architecture.

The implication for the open empirical question of when multi-agent decomposition is justified is that, on this benchmark, the conditions are narrower than the literature on agentic legal AI implies. A practitioner asking whether decomposition helps a CUAD-class extraction task must answer two prior questions: which model family, and which capability tier. On the eleven models tested, only the OpenAI family receives a meaningful multi-agent benefit at the pooled level. That benefit is in turn dominated in F_2/\$ by a B0 deployment on a cheaper Google Flash model.

6.2 The Cascade as a Recall–Precision Exchange

Section 5.5 identifies the classification-gate cascade as the dominant failure mode of Classify-Extract (M0) on the Google family: 97.8–97.9% of M0 false negatives on Gemini 2.5 Flash and Pro originate at the gate rather than at the extractor. The discussion here defends the framing of this as a recall–precision exchange rather than as pure regression.

The classifier reduces over-extraction substantially: mean ω drops from 0.320 under Zero-Shot (B0) to 0.130 under Classify-Extract (M0), a 59% reduction (Table 5.8). Under F_1, which weights precision and recall equally, the gate's gain is sufficient to push M0 to the highest pooled score ($\overline{F_1}$=0.798 vs. B0's 0.737). Under F_2, the gain is insufficient to compensate the recall loss. The cascade reflects an operating-point trade rather than a broken architecture. Under the pre-registered primary metric F_2, the trade is unfavourable for 7 of 11 models. The practitioner question is which operating point matches the deployment cost structure.

The cascade direction is predictable from a single-agent calibration pass. Models with high B0 over-extraction rates (Gemini 2.5 Pro ω_{B0}=0.456; Gemini 2.5 Flash 0.392) inherit a permissive prior on negative instances; the gate amplifies the cautious prior, suppressing recall. Models with low B0 over-extraction (GPT-4.1 Nano ω_{B0}=0.085) gain from the gate because their single-agent extraction loses ground to laziness rather than to over-extraction. The practitioner heuristic is: run B0, inspect ω_{B0}, and deploy M0 only if ω_{B0} is moderate-to-low. Within the present eleven-model panel, models with $\omega_{B0} \lesssim 0.20$ gain on F_1 under M0; those above that threshold do not. The boundary is indicative, not validated.

This result connects to the failure-mode taxonomy of Cemri, Pan, Yang, et al. [Cem+25], who identify the inter-agent verification gap as a leading cause of multi-agent regression: the downstream extractor cannot override the upstream classifier's silent rejection. A reflective verifier agent (absent in M0) would be the natural mitigation, and is identified as an untested architectural pattern in Section 7.4.

6.3 Routing as a Deterministic Lookup

The Multi-Agent (Full) (M1) router agreed with a static category-to-specialist mapping on 99.91% of requests across the panel (Section 5.4). The natural reading is that routing in this design space reduces to a table lookup keyed on the category name, a negative finding for the LLM-router subspace illustrated by ChatLaw v2 [Cui+24] and L-MARS [WY25]. Two qualifications are worth stating.

First, the result depends on three architectural commitments that do not hold in every multi-agent design: the specialist domains were disjoint and descriptively named; the categories carried indicators the router could read directly from the category label; and the partition was fixed before any model was run. In designs where domains overlap, where the category vocabulary is open-ended, or where the routing decision must integrate retrieved context, the router's work becomes non-trivial and routing accuracy falls below ceiling. A second-order caveat is that the LLM router's prompt and the deterministic dictionary it is benchmarked against were authored by the same operator from a single conceptual map of the 41-category schema; the 99.91% agreement therefore measures internal consistency between two artefacts of one design intent, not LLM routing competence on an externally specified taxonomy. The generalisable claim is therefore narrower: in a closed-vocabulary specialist-routing design with an operator-curated category schema, the router's contribution is replicable by a deterministic dictionary at zero LLM cost.

Second, Multi-Agent (Full) (M1) and Multi-Agent (Prompt-Only) (M2) differ statistically on every model despite the routing decision being a near-deterministic dictionary lookup in both cases. Three components vary jointly between the two configurations: the prompt format (M1 receives one focused specialist preamble per call; M2 receives the textual union of three), the dispatch overhead (M1 incurs an orchestrator hop and a router LLM call that M2 does not), and the per-call latency. The Jaccard $+0.071$ pooled advantage for M2 (Table 5.6) is therefore not cleanly attributable to consolidation alone; the most parsimonious reading is attentional. The union prompt presents all three category-cluster framings simultaneously and lets the model choose, rather

than committing the dispatch decision before the contract is read. A router-disabled M1 ablation would be required to isolate the prompt-format component from the dispatch-overhead component (§7.4).

The combined implication for the design of legal-AI multi-agent systems is that specialist prompt content, rather than dispatch architecture, accounts for the M1/M2 delta. A practitioner with a fixed specialist taxonomy is better served by a single agent receiving the union of specialist instructions than by a router-plus-specialists topology, and eliminates the 23% orchestrator cost premium.

6.4 The β=2 Choice and Operating-Point Sensitivity

The pooled architectural ranking depends on the chosen β. At β=1 the ranking is Classify-Extract > Multi-Agent (Full) > Zero-Shot > Multi-Agent (Prompt-Only) > CoT. At β=2 the ranking is Zero-Shot > CoT > Classify-Extract > Multi-Agent (Full) ≈ Multi-Agent (Prompt-Only). The architectural conclusion of the thesis therefore depends, in part, on the β choice.

Setting $F_\beta(\text{M0}) = F_\beta(\text{B0})$ on the pooled precision and recall in Table 5.2 (B0: P=0.666, R=0.854; M0: P=0.813, R=0.793) and solving for β yields a flip-point of $\beta^\star \approx 1.74$ ($\beta^{\star 2} \approx 3.01$). Because β^2 encodes the deployment cost ratio of a missed clause to a false extraction, the empirical interpretation is: B0 dominates M0 on F_β when missing a clause is at least ≈3.0× as costly as flagging a spurious one; M0 dominates below that ratio. The thesis's commitment to β=2 (cost ratio 4:1) sits above the flip-point, which is why the H1 verdict reads as a B0 win; an examiner who prefers $\beta \in [1.0, 1.7]$ (cost ratio between 1:1 and ≈3:1) would read the same data as an M0 win of 0.013 to 0.055 points. Three CLM workflows that plausibly sit in this range are: post-signature obligation tracking (false-positive review cheap relative to a missed renewal-notice deadline), red-line drafting support (reviewer must read every flagged candidate anyway), and compliance screening (false positives feed an existing triage queue). The architectural verdict is therefore conditional on the β choice rather than absolute.

The commitment to β=2 is justified in Section 3.4 by the deployment cost structure of CLM review: a missed clause carries the legal cost of an undisclosed obligation, whereas a spurious extraction is corrected at the lower cost of attorney review time. Deployment cost ratios are out of scope here. The sensitivity is mitigated by reporting F_1, recall, precision, ω, and FN-rate alongside F_2, so a deployment with a different cost ratio can re-rank architectures from the same data. The practitioner recommendation is conditional on β: at β=2 deploy B0; between β=1.0 and $\beta^\star$≈1.74 deploy M0; below β=1

(where false positives are costlier than false negatives, as in some compliance-screening workflows), M0's calibration advantage widens further.

6.5 Benchmark-to-Production Gaps

CUAD measures one axis: extraction accuracy under a fixed forty-one-category taxonomy on clean born-digital text with expert ground truth. This axis is necessary but not sufficient for production deployment. Three deployment axes relocate the architectural argument.

Preprocessing and retrieval are upstream of architecture. CUAD contracts are EDGAR filings: clean, machine-readable, consistently formatted. Enterprise repositories contain scanned PDFs of variable OCR quality, redactions, and partial digitisations; document-preprocessing quality is the primary performance ceiling, set before any extraction architecture is invoked. CUAD's one-to-one (contract, category) framing also omits the retrieve-then-extract pipeline production reviewers actually run, in which a query (e.g. "which contracts contain uncapped liability") is dispatched across the repository before per-contract extraction. The classification-gate cascade (§6.2) is the single-contract analogue of that retrieval failure: the same upstream-rejection dynamics that suppress M0 recall on Gemini 2.5 will plausibly suppress retrieval-extraction recall in production.

Open taxonomies and cross-contract reasoning are unmodelled. CUAD's forty-one categories cover general commercial provisions; construction contracts (variation orders, retention, defects liability), SaaS agreements (SLA penalties, sub-processor notice), and jurisdictional clause variants do not map cleanly onto the schema. The fixed three-domain specialist partition tested here (M1, M2) is not extensible to an open taxonomy without re-routing. Cross-contract questions (consistency of liability caps across vendor agreements; impact analysis on a counterparty acquisition) require portfolio-level orchestration over per-contract agents that no public benchmark currently evaluates and that this thesis identifies as future work (§7.4).

Trace primitives under no ground truth. A production deployment cannot compute F_2, Jaccard, or ω; evaluation proceeds through user feedback, spot-checking, and exception triage. The single architectural property the benchmark does measure that

production review cannot reproduce from a single-agent system is schema-enforced grounding: Multi-Agent (Full)'s 1.000 verbatim-citation rate on every predicted clause (Table 5.9, predicted-clause column) versus 0.543 for Zero-Shot and 0.415 for Multi-Agent (Prompt-Only) means that under M1, every emitted span is locatable in the source contract by exact string match, and under B0/M2 roughly half are paraphrased or hallucinated. This is the one auditability finding the benchmark supports without auxiliary measurement, and is the measured basis for preferring a multi-agent topology in deployments where extraction provenance is a regulatory or contractual requirement (EU AI Act Art. 13 [EC24], GDPR Art. 22 [EC16]). On grounding (the one production-relevant axis the benchmark does measure), multi-agent dominates (Table 5.9).

7 Conclusion

Model identity dominates architectural choice: switching configurations moves pooled F_2 by at most 0.011 points, while moving across the model panel moves it by $\approx$0.4; the family $\times$ configuration interaction explains the residual.

7.1 Summary of Contributions

This thesis evaluated five architectural configurations against eleven frontier proprietary LLMs on the 102-contract CUAD test split, producing 163,644 paired sample-level predictions analysed under a controlled statistical pipeline. The four contributions stated in Chapter 1 are restated here with pointers to the evidence that supports them.

C1: Cross-architecture benchmark. A cross-family cross-architecture comparison on CUAD at this scale: five architectural configurations, eleven frontier models, 55 official runs, analysed with paired McNemar exact tests, paired bootstrap confidence intervals, and Benjamini–Hochberg correction. The benchmark, prompt registry, run artefacts, and statistical pipeline are released as open source at `https://github.com/ndtrung00/agentic-ai-for-clm` (Section 4.7).

C2: Capability-tier gradient. Architectural ranking is not constant across the model panel. Pooled F_2 across the five configurations spans a 0.011-point band and per-model winners are split four ways (Table 5.3). The capability-tier breakdown (Figure 5.2) reveals the gradient: light- and mid-tier models gain from Classify-Extract's (M0) calibration; heavy-tier models are dominated by Zero-Shot (B0). No architecture is universally optimal across the eleven-model frontier-proprietary panel evaluated here; the choice is conditional on model family, capability tier, and the chosen β of the F_β metric.

C3: Classification-gate cascade. The pipeline-decomposition pattern (Classify-Extract, M0) suffers a recall–precision exchange mediated by the classification gate. On Gemini 2.5 Flash and Pro, 97.8–97.9% of M0 false negatives originate at the gate rather than at the extractor (Table 5.7). The exchange's direction is predictable from a single-agent calibration pass: models with high Zero-Shot (B0) over-extraction lose recall to the gate; models with low B0 over-extraction gain F_1 from it. Section 6.2 formalises the heuristic and connects it to the inter-agent verification gap taxonomy of Cemri, Pan, Yang, et al. [Cem+25].

C4: Routing redundancy with span-quality and grounding tradeoffs. The LLM router agreed with a deterministic category-to-specialist dictionary on 99.91% of requests across the panel and on 100% for ten of eleven models. Multi-Agent (Prompt-Only) (M2) matches Multi-Agent (Full) (M1) on F_2 pooled to three decimals and produces tighter spans (Jaccard $+0.071$ on ten of eleven models; Table 5.6). The architectural advantage that does survive ablation is grounding: Multi-Agent (Full)'s schema-enforced specialist output achieves 100% verbatim citation of contract text on every predicted clause, against 54% for Zero-Shot and 42% for Multi-Agent (Prompt-Only) (Table 5.9). The LLM-mediated routing call adds orchestrator cost without measurable F_2 benefit, but the specialist scaffold to which the router dispatches carries the auditability surplus.

7.2 Answers to the Research Questions

Main RQ. *Can a multi-agent framework improve contract clause extraction accuracy beyond single-agent baselines while providing superior explainability?* On accuracy (F_2), multi-agent decomposition is dominated at the pooled level by Zero-Shot (B0), and conditionally improves only on the OpenAI family and on lighter capability tiers. On efficiency (cost-adjusted F_2), multi-agent configurations are Pareto-dominated by B0 deployments on cheaper models (Figure 5.5). On explainability, the pipeline (Classify-Extract, M0) and routed-specialist (Multi-Agent (Full), M1) topologies produce the trace primitives that production review needs (Section 4.6), which is the axis on which the practitioner argument for multi-agent architectures rests (Section 6.5).

RQ1: Architectural pattern by condition. No pattern wins universally. Pooled F_2 ranks Zero-Shot > CoT > Classify-Extract > Multi-Agent (Full) $\approx$ Multi-Agent (Prompt-Only) within a 0.011-point band. The model-family interaction is the dominant signal:

the Anthropic family peaks under CoT (B1), the Google family under Zero-Shot (B0), and the OpenAI family under Classify-Extract (M0). The prevalence-tier interaction is also large: M1 specialists lead on common-tier categories, M0 on moderate, and B0 on rare (Figure 5.3). The capability-tier interaction completes the picture: Classify-Extract helps light and mid models, Zero-Shot helps heavy models. The answer to RQ1's "under what conditions" clause is therefore: M0 is preferable when the model is OpenAI-family or light-tier, F_1 is the primary metric, and the practitioner accepts the 1.67× cost premium.

RQ2: Multi-agent vs. single-agent on accuracy and cost. Multi-agent configurations carry mean cost premiums of 1.23× (Multi-Agent (Full), M1) to 1.67× (Classify-Extract, M0) over Zero-Shot (B0) without compensating F_2 gains pooled across the panel. Zero-Shot occupies the Pareto frontier at every cost level (Figure 5.5). The only multi-agent cost premium that pays back in F_2 terms is M0 on the OpenAI family, and even that is dominated in F_2/\$ by a B0 deployment on Gemini 2.5 Flash. On F_1, Classify-Extract leads pooled and the premium is partially justified for precision-critical deployments. This Pareto-dominance result is conditional on per-call provider billing without prompt caching. Caching, supported by every provider in the panel, would shrink the M0/M1 premium toward the orchestrator-hop overhead alone and may invert the frontier on long-contract repeated-category workloads where the system prompt and contract text are amortised across many extraction calls (§7.4); under that cost model, RQ2 is open.

RQ3: Auditable reasoning traces. The parent–child trace structure of Classify-Extract (M0) and Multi-Agent (Full) (M1) produces decision attribution (routing decision, specialist extraction, validation outcome) that Zero-Shot (B0) cannot (Section 4.6). Trace completeness is 100% on B0/B1/M2/M0 and 95.2% on M1, with the residual 4.8% attributable to provider-side truncation on long contracts (§4.6). The trace structure is paired with schema-enforced grounding: Multi-Agent (Full) reaches 100% verbatim contract-text citation on every predicted clause and Classify-Extract reaches 89%, against 54% for Zero-Shot and 42% for Multi-Agent (Prompt-Only) (Table 5.9).

Three practitioner-relevant tradeoffs attend this auditability surplus. *Storage cost* scales approximately 4× per M1 call (orchestrator hop plus three specialist sub-prompts logged at inference time). *Attribution faithfulness* is bounded by the trace format: the record is faithful to the call graph but does not certify the model's internal computation. *Routing redundancy* adds latency without decision diversity: the 99.91% deterministic routing result means the orchestrator layer can be replaced by a static dictionary at zero LLM cost.

7.3 Limitations

The limitations below bound the generalisation of these findings; they expand the threats formalised in Section 3.7.

Single benchmark. The evaluation is on the 102-contract CUAD test split under the upsampled-positive class distribution (§3.2). The findings may not extrapolate to non-CUAD clause taxonomies, to non-English contracts, or to non-U.S. jurisdictions. The contract-length moderation result (Section 5.8, Table 5.11) shows that the architectural ranking is not stable across document lengths within CUAD itself: multi-agent helps on short contracts and hurts on long ones. The architectural ranking observed here is therefore a ranking on this evaluation envelope, not a universal ranking.

Test-set contamination. CUAD [Hen+21] has been publicly hosted on HuggingFace since 2021 and is plausibly present in the pre-training data of every model in the panel; benchmark contamination is a documented threat to the construct validity of LLM evaluations [Sai+23; MS22]. The present evaluation does not separate extraction skill from memorisation, and the F_2 point estimates may overstate true extraction competence. Two observations partially constrain the threat. First, the architectural ranking varies systematically across model families with different documented training cut-offs (Anthropic peaks under CoT, Google under Zero-Shot, OpenAI under Classify-Extract; §6.1), which uniform memorisation would not predict. Second, the M0 classification-gate cascade (§6.2) is a recall-loss mechanism (silent upstream rejection on positive instances) that contamination cannot generate; memorised content would, if anything, raise recall rather than collapse it. A held-out non-CUAD evaluation (e.g. a clause subset from LegalBench or ContractNLI [KM21]) is identified in Section 7.4 as the highest-priority extension.

Pinned model snapshots. Model identifiers are pinned to specific snapshots in the run metadata. Provider behaviour can shift under silent model updates; the family interaction observed here may not be stable across future provider releases. The qualitative pattern is consistent with findings reported across task domains in the recent literature [Gao+25; Cem+25]: architecture choice interacts with model family more than with any within-architecture variable, suggesting the pattern generalises even where specific point estimates do not.

Closed proprietary panel. Open-weights models are excluded (Section 3.3). The proprietary–open-source gap reported by S. Liu, Z. Li, Ma, et al. [Liu+25] is not re-examined here. The finding that multi-agent decomposition does not improve pooled F_2 may not transfer to a panel of weaker open-weights models, where decomposition's capability-equalisation effect (Section 6.1) might be larger. Open-weights extension is identified in Section 7.4 as the most consequential extension of the present work.

7.4 Future Work

Three axes under-determine the conclusion: untested architectural patterns, evaluation beyond CUAD, and cost mechanisms not exercised in the experiments.

Untested architectural patterns. Three patterns absent from the five-configuration grid are expected to behave differently. Reflective verifier agents [Shi+23] could absorb the classification-gate cascade (§6.2) by letting the extractor override a silent rejection on confidence-flagged borderline cases. Tool-calling specialists that retrieve clause exemplars or jurisdictional indicators before extracting extend the design to open-taxonomy deployments (§6.5). Hierarchical orchestration (a portfolio-level orchestrator over per-contract extraction agents) targets cross-contract reasoning, which is not exercised by any current public benchmark.

Beyond CUAD. Four extensions widen the evaluation envelope: open-weights models, where the proprietary–open-source gap of S. Liu, Z. Li, Ma, et al. [Liu+25] may amplify M0's capability-equalisation effect and invert the architectural ranking; retrieve-then-extract on a multi-contract corpus, the production-relevant generalisation of M0; document-quality stress tests (OCR noise, redactions, mixed-format inputs) that probe whether the ranking survives preprocessing degradation; and formal user studies with practising reviewers, which would close the loop on RQ3 by measuring whether M0/M1 trace primitives actually shorten review time. The thesis treats this property as a system feature, not as an outcome.

Cost mechanisms. Two cost mechanisms not exercised here would re-form the Pareto frontier of §5.7. Prompt caching, supported by every provider in the panel, would cut the marginal cost of per-category re-prompting that dominates the B0 cost structure

on long contracts and shrink the 23–67% multi-agent premium. Structured-output decoding would reduce M0 classifier-call latency to where the cascade exchange becomes acceptable on more models. The cost ranking under caching and constrained decoding remains open.

On accuracy alone, the slice tested here does not justify multi-agent deployment over a strong single-agent baseline. The case for multi-agent architectures rests on axes the benchmark does not measure: auditability, open-taxonomy extensibility, and cross-contract reasoning. Future work should evaluate those axes directly.

List of Figures

2.1 CLM lifecycle. Clause extraction, risk identification, and obligation tracking (highlighted) occur primarily in the Execute and Monitor stages. 6

2.2 Multi-agent design space: five structural axes and their principal options. Shaded boxes mark options covered by at least one experimental configuration (B0, B1, M0, M1, M2) in this thesis. Chapter 3 specifies which configuration selects which option. 8

2.3 Per-category F_1 for 13 representative CUAD categories under the zero-shot baseline (B0), pooled across the 11-model panel (§3.3); error bars span $\pm 1\sigma$ across models. The rare-tier categories combine *lower mean* F_1 with *wider variance across models* (σ up to 0.17), the empirical pattern that motivates the per-domain specialist design evaluated in Chapter 3. . . . 10

3.1 Message-flow diagrams for the five configurations. Input = contract text + clause-category indicator. Blue nodes: generalist LLM invocations. Orange nodes: domain-specialist prompts. $\emptyset$ = silent no-clause return. 18

4.1 System component diagram. *Solid-border boxes*: code components (Loader, StateGraph, ModelClient, Statistical Pipeline, Dashboard). *Dashed boxes*: data stores. *Orange boxes*: external provider APIs. *Dashed arrows*: control flow (Driver initialises inputs before each run). *Solid arrows*: data flow. Only the StateGraph topology varies across B0–M2; all other components are configuration-agnostic. 27

4.2 M1 sequence diagram for a single (contract, category) invocation. The router call is the Langfuse parent; the specialist call is its child. 32

5.1 Mean F_2 by provider family $\times$ configuration. Each family peaks under a different configuration: Anthropic under CoT (B1, 0.851), Google under Zero-Shot (B0, 0.826), OpenAI under Classify-Extract (M0, 0.783). No single architecture is universally optimal. The family-by-configuration interaction is the dominant signal in the H1 results. 39

5.2 Capability tier × configuration heatmap of mean F_2. Cells coloured by $\overline{F_2}$; orange outline marks the best configuration per tier. Classify-Extract (M0) wins on Light and Mid tiers; Zero-Shot (B0) wins on Heavy. Decomposition narrows the Heavy–Light capability gap from 0.106 under B0 to 0.054 under M0 (49% reduction); the same decomposition penalises Heavy models on F_2 because they already absorb long-context load in a single call (see §6.1). 40

5.3 Mean F_2 by CUAD prevalence tier × configuration, pooled across the eleven-model panel. Tier partition derived from positive-instance prevalence in the 102-contract test split (common > 30% positive, moderate 10–30%, rare < 10%); marginally different from the ContractEval F1-based partition (6/14/21) without changing the qualitative ranking. The best configuration per tier differs: M1 wins common (0.935), M0 wins moderate (0.780), B0 and B1 tie on rare (0.658). Multi-agent decomposition helps on categories where the base model is already strong; H2 is rejected because it predicts the opposite direction for rare categories. . . 44

5.4 Recall–precision exchange under Classify-Extract. Each B0 model (•) maps to its M0 counterpart (▲); arrows show the per-model trajectory in the (ω, λ) plane. Most models move *left and slightly up*: a moderate precision gain (lower ω) paid with a small recall loss (higher λ). On Gemini 2.5 Flash and Pro (orange arrows) the recall loss is severe: λ jumps to > 0.32, the cascade documented in Table 5.7. On GPT-4.1 Nano (green arrow) the trajectory reverses: the gate corrects the model's high B0 laziness (λ=0.555 B0 → 0.363 M0). The dotted diagonal marks $\omega = \lambda$ as an orientation reference. 48

5.5 F_2 vs. per-sample cost across all 55 cells. The Pareto frontier is occupied by B0 at every cost level. The three Gemini M0 cells (Gemini 2.5 Flash, Gemini 2.5 Pro, and Gemini 3 Flash) are strictly Pareto-dominated: lower F_2 *and* higher cost than B0 alternatives. 50

List of Tables

2.1 Frontier LLM releases referenced in this thesis. Context-window figures denote the standard production context; beta long-context modes noted parenthetically. Capabilities summarised from official announcements. 7
2.2 CUAD dataset statistics [Hen+21]. 9
2.3 Main contract-analysis benchmarks. 11
2.4 Multi-agent legal AI systems coded against the five design-space axes of Section 2.3. All prior systems evaluate a single architecture without ablation. 12

3.1 Five evaluated configurations. B1 vs. B0 isolates chain-of-thought; M0 vs. B0 isolates the classification gate; M1 vs. M2 isolates routing and specialist decomposition (architecture ablation). †Artifact IDs used in the experiment repository: B0 = B1, B1 = B4, M0 = M0, M1 = M1, M2 = M6. . 17
3.2 Eleven frontier proprietary models evaluated. Three families; compute-tier range covers light-to-heavy within each family. Snapshot IDs are the exact provider model strings used in all API calls. 20

4.1 Category-to-specialist assignment used by M1. All 41 CUAD categories are listed; the mapping is fixed before any model is run. 31

5.1 Coverage: analysed predictions per (model, configuration) cell after dropping malformed records. *Partial run due to upstream sampling interruption; paired tests use the sample-ID intersection. Configuration names defined in Table 3.1. The two B0 partial cells are not missing-completely-at-random: GPT-4.1's 471 dropped records concentrate on five long contracts (backfilled median length ≈122,000 characters vs. ≈24,000 for retained records), consistent with provider-side API time-outs on the long-context tail; Sonnet 4's 359 dropped records spread across 98 of 102 contracts with a milder length skew (median ≈31,000 vs. ≈26,000). The contract-length moderation analysis (Section 5.8) shows multi-agent configurations underperform B0 on long contracts; for GPT-4.1 the missing-data direction therefore biases the Tab 5.4 ΔF_2 *toward* the multi-agent configurations and the per-model H1 verdict for GPT-4.1 understates rather than overstates the multi-agent disadvantage on the full evaluation envelope. 37
5.2 Headline metrics by family and configuration (Cfg), mean across constituent models. ω = over-extraction rate (FP / negatives). Bold: best-in-column per family. Configuration names defined in Table 3.1. The F_2 and F_1 rankings diverge: pooled F_2 favours Zero-Shot (B0, 0.801) by 0.011 points; pooled F_1 favours Classify-Extract (M0, 0.798) by 0.061 points. 38
5.3 F_2 by model and configuration. Bold: best per row. Configuration names defined in Table 3.1. The family interaction is pronounced: the Anthropic family peaks under CoT (B1), the Google family under Zero-Shot (B0), and the OpenAI family under Classify-Extract (M0). 40
5.4 H1 paired tests: $\Delta F_2 = F_2(\text{multi}) - F_2(\text{B0})$ computed on the sample-ID intersection (so the row ΔF_2 may differ from the full-run difference implied by Table 5.3 when one of the pair is a partial run, namely Claude Sonnet 4 and GPT-4.1). Paired-bootstrap 95% CIs (B=300, `seed=42`), BH-corrected significance (α=0.05, m=22). ✓: BH-reject H_0. Note: BH significance is decided by the McNemar exact p-value; the bootstrap CI on ΔF_2 is reported separately and may not align with the McNemar verdict on near-zero effects where the discordant-pair distribution and the resampled-mean distribution diverge. 41
5.5 Rare–common gap $g = F_2^c - F_2^r$ per (model, configuration). Lower is better. Classify-Extract's mean gap (0.284) exceeds Zero-Shot's (0.276) due to gate-induced rare-tier collapse on Gemini 2.5 Flash/Pro. Multi-Agent (Full)'s mean gap (0.342) is worst, driven by GPT-family specialist failure on rare categories. Only CoT and Multi-Agent (Prompt-Only) reduce the gap relative to Zero-Shot (means 0.240 and 0.226). 42

5.6 H3 contrasts: Multi-Agent (Full) (M1) vs. Multi-Agent (Prompt-Only) (M2). Both configurations are tied on pooled F_2 ($\overline{\Delta F_2} = -0.0002$; 5 M1 wins, 6 M2 wins). On span quality (Jaccard), Multi-Agent (Prompt-Only) dominates on 10 of 11 models, mean advantage +0.071. BH: BH-corrected significance (α=0.05, m=11). All 11 contrasts are significant. The single M1 exception on Jaccard is Gemini 3 Flash, where Multi-Agent (Prompt-Only) collapses (Section 5.5). 45
5.7 M0 classification-gate rejection cascade. FN-rate = $\mathrm{FN}/(\mathrm{TP}+\mathrm{FN})$. Gate-FN: false negatives where the classifier emitted `predicted_no_clause`. Gate % = Gate-FN / M0 FN. On average 82.7% of M0's false negatives originate at the gate; Gemini 2.5 Flash and Pro lose ≥97.8% of their recall to upstream rejection. 46
5.8 Over-extraction rate ω (Eq. 2.5): fraction of negative-label instances on which the model emits a non-empty span. Bold: best per row. The classification gate cuts ω by more than half on average; the Gemini family gains the most precision at the recall cost documented in Table 5.7. CoT (B1) raises ω above Zero-Shot (B0) on every model, evidence that chain-of-thought prompting amplifies over-extraction; Multi-Agent (Full) (M1) stays close to Classify-Extract (M0) because it inherits the same specialist scaffold; Multi-Agent (Prompt-Only) (M2) reverts to the free-text baseline. 47
5.9 Mean grounding rate by configuration. *All*: pooled over all ≈ 2,999 samples per cell, including "no related clause" returns scored 0. *Predicted-clause only*: conditioned on a non-empty prediction. Multi-Agent (Full) (M1) achieves 1.000 pooled across every model in the predicted-clause column: the structured specialist output enforces contract-text citation. Classify-Extract (M0) reaches 0.893. Zero-Shot (B0) and Multi-Agent (Prompt-Only) (M2), which return free-text extractions, leave 44–58% of predicted spans ungrounded. 49
5.10 Provider cost per run (≈2,999 samples) and per-model mean ratio relative to B0. M0's 1.67× premium reflects the additional classifier call; M1's 1.23× reflects the orchestrator hop. 50
5.11 Mean F_2 by contract-length quartile (102 contracts, ≈26 per quartile), pooled across all eleven models. Quartile boundaries: 14,384 / 25,922 / 62,272 characters. Multi-agent configurations (M0, M1) outperform B0 on short contracts (Q1–Q2) by +1.7 to +3.7 percentage points and underperform on long contracts (Q3–Q4) by −0.5 to −4.1 percentage points. The pattern is opposite to the context-window motivation. 51

Bibliography

[Ant24] Anthropic. *Introducing the Model Context Protocol*. Anthropic News Release. Released 25 November 2024. Nov. 2024.

[AO 23] A&O Shearman. *A&O announces exclusive launch partnership with Harvey*. Press Release. Feb. 2023.

[BH95] Y. Benjamini and Y. Hochberg. "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing." In: *Journal of the Royal Statistical Society, Series B (Methodological)* 57.1 (1995), pp. 289–300. DOI: 10.1111/j.2517-6161.1995.tb02031.x.

[BSS25] T. Bogavelli, H. Subramani, and R. Sharma. *AgentArch: A Comprehensive Benchmark to Evaluate Agent Architectures in Enterprise*. 2025. arXiv: 2509.10769.

[Car+20] D. Card, P. Henderson, U. Khandelwal, R. Jia, K. Mahowald, and D. Jurafsky. "With Little Power Comes Great Responsibility." In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020)*. 2020, pp. 9263–9274. DOI: 10.18653/v1/2020.emnlp-main.745. arXiv: 2010.06595.

[Cem+25] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica. *Why Do Multi-Agent LLM Systems Fail?* 2025. arXiv: 2503.13657.

[Cha+20] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos. "LEGAL-BERT: The Muppets Straight Out of Law School." In: *Findings of the Association for Computational Linguistics: EMNLP 2020*. 2020, pp. 2898–2904. arXiv: 2010.02559.

[Cha+24] A. Chatterjee, H. S. V. N. S. K. Renduchintala, S. Bhatia, and T. Chakraborty. "POSIX: A Prompt Sensitivity Index for Large Language Models." In: *Findings of the Association for Computational Linguistics: EMNLP 2024*. 2024, pp. 14550–14565. arXiv: 2410.02185.

[Coh88] J. Cohen. *Statistical Power Analysis for the Behavioral Sciences*. 2nd. Hillsdale, NJ: Lawrence Erlbaum Associates, 1988. ISBN: 0-8058-0283-5.

[Con87] R. J. Connor. "Sample size for testing differences in proportions for the paired-sample design." In: *Biometrics* 43.1 (1987), pp. 207–211. DOI: 10.2307/2531961.

[Cui+24] J. Cui, M. Ning, Z. Li, B. Chen, Y. Yan, H. Li, B. Ling, Y. Tian, and L. Yuan. *ChatLaw: A Multi-Agent Collaborative Legal Assistant with Knowledge Graph Enhanced Mixture-of-Experts Large Language Model*. Version 2 (2024); v1 (2023) is the original retrieval-augmented system. 2024. arXiv: 2306.16092.

[Dev+19] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2019)*. 2019, pp. 4171–4186. arXiv: 1810.04805.

[Dom+24] R. Dominguez-Olmedo, V. Nanda, R. Abebe, S. Bechtold, C. Engel, J. Frankenreiter, K. Gummadi, M. Hardt, and M. Livermore. *Lawma: The Power of Specialization for Legal Annotation*. ICLR 2025. 2024. arXiv: 2407.16615.

[Dro+18] R. Dror, G. Baumer, S. Shlomov, and R. Reichart. "The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing." In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL 2018)*. 2018, pp. 1383–1392.

[EC16] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)*. Official Journal of the European Union, L 119, 4.5.2016, pp. 1–88. https://eur-lex.europa.eu/eli/reg/2016/679/oj. 2016.

[EC24] European Parliament and Council of the European Union. *Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*. Official Journal of the European Union, L 2024/1689, 12.7.2024. http://data.europa.eu/eli/reg/2024/1689/oj. 2024.

[Efr79] B. Efron. "Bootstrap Methods: Another Look at the Jackknife." In: *The Annals of Statistics* 7.1 (1979), pp. 1–26. DOI: 10.1214/aos/1176344552.

[Gao+25] M. Gao, Y. Li, B. Liu, Y. Yu, P. Wang, C.-Y. Lin, and F. Lai. *Single-agent or Multi-agent Systems? Why Not Both?* 2025. arXiv: 2505.18286.

[Har+20] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant. "Array Programming with NumPy." In: *Nature* 585 (2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2.

[Hat+23] J. Hatzius, J. Briggs, D. Kodnani, and G. Pierdomenico. *The Potentially Large Effects of Artificial Intelligence on Economic Growth (Briggs/Kodnani)*. Goldman Sachs Global Economics Analyst. Published 26 March 2023. Primary report is paywalled; widely cited in *The Globe and Mail* and *Law.com*. Mar. 2023.

[Hen+21] D. Hendrycks, C. Burns, A. Chen, and S. Ball. "CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review." In: *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS 2021)*. 2021. arXiv: 2103.06268.

[Hon+24] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber. "MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework." In: *12th International Conference on Learning Representations (ICLR 2024)*. Oral presentation. 2024. arXiv: 2308.00352.

[KM21] Y. Koreeda and C. D. Manning. "ContractNLI: A Dataset for Document-level Natural Language Inference for Contracts." In: *Findings of the Association for Computational Linguistics: EMNLP 2021*. 2021, pp. 1907–1919. arXiv: 2110.01799.

[Lam+16] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer. "Neural Architectures for Named Entity Recognition." In: *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2016)*. 2016, pp. 260–270. arXiv: 1603.01360.

[Lan24a] LangChain Inc. *LangGraph: Building Stateful, Multi-Actor Applications with LLMs*. Software library. https://github.com/langchain-ai/langgraph. 2024.

[Lan24b] Langfuse GmbH. *Langfuse: Open-Source LLM Engineering Platform*. Software / documentation. https://langfuse.com/ ; https://github.com/langfuse/langfuse. 2024.

[Li+23] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem. "CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society." In: *Advances in Neural Information Processing Systems (NeurIPS 2023)*. 2023. arXiv: 2303.17760.

[Liu+25] S. Liu, Z. Li, R. Ma, H. Zhao, and M. Du. *ContractEval: Benchmarking LLMs for Clause-Level Legal Risk Identification in Commercial Contracts*. 2025. arXiv: 2508.03080.

[LMP01] J. D. Lafferty, A. McCallum, and F. C. N. Pereira. "Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data." In: *Proceedings of the 18th International Conference on Machine Learning (ICML 2001)*. 2001, pp. 282–289.

[Mar+24] L. Martin, N. Whitehouse, S. Yiu, L. Catterson, and R. Perera. *Better Call GPT: Comparing Large Language Models Against Lawyers*. Onit AI Centre of Excellence. 2024. arXiv: 2401.16212.

[McN47] Q. McNemar. "Note on the Sampling Error of the Difference between Correlated Proportions or Percentages." In: *Psychometrika* 12.2 (1947), pp. 153–157. DOI: 10.1007/BF02295996.

[MS22] I. Magar and R. Schwartz. "Data Contamination: From Memorization to Exploitation." In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Dublin, Ireland: Association for Computational Linguistics, 2022, pp. 157–165. DOI: 10.18653/v1/2022.acl-short.18. arXiv: 2203.08242.

[PwC23] PwC. *PwC partners with OpenAI and Harvey to build domain specific foundation models*. PwC Press Release. 2023.

[Rap+25] P. Raptopoulos, G. Filandrianos, M. Lymperaiou, and G. Stamou. "PAKTON: A Multi-Agent Framework for Question Answering in Long Legal Agreements." In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP 2025)*. Oral presentation. 2025. arXiv: 2506.00608.

[RM05] S. Riezler and J. T. Maxwell III. "On Some Pitfalls in Automatic Evaluation and Significance Testing for MT." In: *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*. 2005, pp. 57–64.

[Sai+23] O. Sainz, J. A. Campos, I. García-Ferrero, J. Etxaniz, O. Lopez de Lacalle, and E. Agirre. "NLP Evaluation in trouble: On the Need to Measure LLM Data Contamination for each Benchmark." In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. Singapore: Association for Computational Linguistics, 2023, pp. 10776–10787. DOI: 10.18653/v1/2023.findings-emnlp.722. arXiv: 2310.18018.

[San25] M. Sanwal. *Layered Chain-of-Thought Prompting for Multi-Agent LLM Systems: A Comprehensive Approach to Explainable Large Language Models*. 2025. arXiv: 2501.18645.

[SF12] G. M. Sullivan and R. Feinn. "Using Effect Size—or Why the P Value Is Not Enough." In: *Journal of Graduate Medical Education* 4.3 (2012), pp. 279–282. DOI: 10.4300/JGME-D-12-00156.1.

[Shi+23] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. "Reflexion: Language Agents with Verbal Reinforcement Learning." In: *Advances in Neural Information Processing Systems (NeurIPS 2023)*. 2023. arXiv: 2303.11366.

[SMP24] K. Sommers, K. McDonald, and L. Phelan. *Magic Quadrant for Contract Life Cycle Management*. Tech. rep. Published 14 October 2024. Gartner, Oct. 2024.

[Sur+25] R. Surapaneni et al. *Announcing the Agent2Agent Protocol (A2A)*. Google Developers Blog. Protocol subsequently transferred to the Linux Foundation, June 2025. Apr. 2025.

[The20] The pandas development team. *pandas-dev/pandas: Pandas*. Zenodo. Version 2.2.x; see https://zenodo.org/records/3509134 for release-specific DOIs. 2020. DOI: 10.5281/zenodo.3509134.

[Tug+20] D. Tuggener, P. von Däniken, T. Peetz, and M. Cieliebak. "LEDGAR: A Large-Scale Multi-label Corpus for Text Classification of Legal Provisions in Contracts." In: *Proceedings of the 12th Language Resources and Evaluation Conference (LREC 2020)*. 2020, pp. 1235–1241.

[Vir+20] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors. "SciPy 1.0: Fundamental

Algorithms for Scientific Computing in Python." In: *Nature Methods* 17 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.

[Wan+23a] S. H. Wang, A. Scardigli, L. Tang, W. Chen, D. Levkin, A. Chen, S. Ball, T. Woodside, O. Zhang, and D. Hendrycks. "MAUD: An Expert-Annotated Legal NLP Dataset for Merger Agreement Understanding." In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*. 2023, pp. 16369–16382. arXiv: 2301.00876.

[Wan+23b] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. "Self-Consistency Improves Chain of Thought Reasoning in Language Models." In: *11th International Conference on Learning Representations (ICLR 2023)*. 2023. arXiv: 2203.11171.

[Wan+24] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou. *Mixture-of-Agents Enhances Large Language Model Capabilities*. Also published at ICLR 2025. 2024. arXiv: 2406.04692.

[Wei+22] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." In: *Advances in Neural Information Processing Systems (NeurIPS 2022)*. 2022. arXiv: 2201.11903.

[Wor25] World Commerce & Contracting. *Contract Management: An Overlooked Driver of Business Agility and Financial Performance*. WorldCC Whitepaper. Available at https://www.worldcc.com/Portals/IACCM/Reports/Contract%20Management%20Whitepaper.pdf. 2025.

[Wu+23] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework*. Also published at COLM 2024. 2023. arXiv: 2308.08155.

[WY25] Z. Wang and B. Yuan. *L-MARS: Legal Multi-Agent Workflow with Orchestrated Reasoning and Agentic Search*. 2025. arXiv: 2509.00761.

[Yao+23a] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. "Tree of Thoughts: Deliberate Problem Solving with Large Language Models." In: *Advances in Neural Information Processing Systems (NeurIPS 2023)*. 2023. arXiv: 2305.10601.

[Yao+23b] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. "ReAct: Synergizing Reasoning and Acting in Language Models." In: *11th International Conference on Learning Representations (ICLR 2023)*. 2023. arXiv: 2210.03629.